SaG-Prescribe: Unifying Diagnostic Pathways with Counterfactual Refactoring for Automated Code Review and Software Quality Evaluation in Distributed Publish–Subscribe Systems

Ibrahim Onuralp Yigit*

Feza Buzluca†

[DEPARTMENT], [INSTITUTION]
[STREET ADDRESS], [CITY], [POSTAL CODE], [COUNTRY]

November 4, 2026

Abstract

Automated code review relies largely on file- or class-level static analysis. In publish–subscribe architectures (ROS 2, DDS, MQTT), loose coupling hides indirect dependencies, creating an **Architecture-Code Gap**: clean unit-level metrics can coexist with single points of failure, co-location bottlenecks, and transport contract mismatches. We present **SaG-Prescribe**, pairing a deterministic **Diagnostic Pathway** with counterfactually verified refactoring. The pathway extracts a multi-layer dependency graph from Architecture-as-Code descriptors, folds code-level metrics into a **Code Quality Penalty**, computes a hierarchical **Reliability–Maintainability (RM)** model informed by ISO/IEC 25010 and 25019, and runs a catalog of nineteen publish–subscribe anti-patterns. The prescriptive engine compiles findings into three graph-mutation operators—topic splitting, anti-affinity reallocation, QoS hardening—admitting an edit only when its cascade-impact reduction exceeds simulator seed noise at every threshold.

Across eight synthetic scenarios (29–520 vertices, 12–300 applications) we report calibration evidence an accuracy-framed study would omit. Ranking saturates: the RM composite ($\rho = 0.485$) and degree centrality (0.519) are statistically indistinguishable (paired $p = 0.95$). The anti-pattern catalog is indiscriminate on these generated topologies ($\kappa = -0.002$, implicating 94.8% of components) but discriminates on three architectures transcribed from open-source systems ($\kappa = +0.296$, implicating 56%): the negative result is corpus-specific. Availability is degenerate on the application-layer projection. Counterfactual verification admits 1128 of 1589 candidates (71.0%); all seven scenarios improve on the framework's own risk index, but only five do under the independent cascade oracle (mean +3.5%, $p = 0.22$), so the prescriptive effect is promising rather than significant. We state these limits explicitly, alongside a re-measurement of our own prior published correlation.

Keywords: Automated code review; Software quality evaluation; Architectural anti-patterns; Refactoring recommendation; Publish–subscribe middleware; Counterfactual verification.

1 Introduction

1.1 Context and Motivation: The Need for Architectural Code Review

Automated code review and software quality evaluation have become indispensable pillars of modern software engineering (Tufano et al., 2022). As distributed software systems scale in

*Corresponding author. E-mail: [CORRESPONDING-AUTHOR-EMAIL]. ORCID: [0000-0000-0000-0000].
†ORCID: [0000-0000-0000-0000].

complexity and release cycles compress under continuous integration and continuous delivery (CI/CD) regimes, automated tools must detect regressions, evaluate non-functional requirements, and recommend actionable remediations before defective changes reach production fabrics.

Distributed publish–subscribe middleware frameworks—including the Robot Operating System (ROS 2), the Object Management Group's Data Distribution Service (DDS), and MQTT—form the communication infrastructure of contemporary microservices, IoT smart cities, and safety-critical cyber-physical systems (Lee et al., 2025b,a). These frameworks decouple message producers and consumers across space, time, and synchronization via topic-based publish–subscribe channels and message brokers. However, this architectural decoupling introduces an insidious challenge for automated quality assurance: it obscures the end-to-end dependency structure of the system. Failures, buffer overflows, and latency spikes propagate along indirect topological pathways that are completely invisible when analyzing source code files in isolation.

1.2 Two Open Gaps: The Architecture-Code Gap and Open-Loop Review

Quality evaluation in continuous integration has historically relied on Static Code Analysis (SCA) platforms (e.g., SonarQube, SpotBugs, ESLint). This creates a fundamental **Architecture-Code Gap**:

1. **The Architecture-Code Gap:** A distributed system can achieve pristine code quality within every individual component—clean class hierarchies, zero file-level code smells, and low cyclomatic complexity—yet remain brittle at the architectural topology level. Vulnerabilities such as single points of failure (SPOFs), severe topic fan-out bottlenecks, co-located deployment risks, and incompatible Quality of Service (QoS) transport contracts cannot be detected by examining source files in isolation. Shifting structural dependability checks left into the development lifecycle demands a transition from traditional Static Code Analysis to *Static System Analysis (SSA)*.

2. **The Unnamed-Pathology and Open-Loop Review Gap:** In object-oriented programming, mature catalogs of bad smells and anti-patterns (e.g., God Class, Feature Envy, Shotgun Surgery) provide developers with a shared vocabulary, testable detection rules, and proven refactoring patterns (Fowler, 1999; Brown et al., 1998; Suryanarayana et al., 2014). Microservices research has similarly begun cataloging smells (e.g., distributed monoliths, cyclic dependencies, shared databases) (Taibi et al., 2020), and software performance engineering has long catalogued antipatterns of resource use (Smith and Williams, 2000; Trubiani et al., 2023). Neither literature states its patterns over the constructs through which publish–subscribe systems fail—topics, brokers, and QoS contracts—and, to our knowledge, *publish–subscribe architectures lack an equivalent formal anti-pattern catalog stated over those constructs*. Consequently, architectural flaws are discovered reactively through production cascade postmortems rather than proactively during code review. Moreover, existing topology-aware diagnostic frameworks typically operate *open-loop*: they compute abstract numerical centrality scores or black-box failure-impact predictions without identifying the specific architectural pathology at fault or synthesizing verified, actionable refactoring guidance (Harman et al., 2012; Aleti et al., 2013).

1.3 The Diagnostic Pathway: Transparent, Explainable Software Quality Evaluation

To address these gaps, this paper brings the **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation in distributed systems. We present **SaG-Prescribe**, a comprehensive framework that integrates deterministic diagnostic quality attribution, anti-pattern detection, and closed-loop prescriptive refactoring within continuous integration workflows.

SaG-Prescribe decouples quality assurance into two principled, cooperative pathways (Figure 1):

- **The Diagnostic Pathway (Deterministic Software Quality Evaluation & Attribution):** The Diagnostic Pathway operates statically on Architecture-as-Code descriptors. It constructs a multi-layer heterogeneous dependency graph (G_{analysis}) encompassing applications, libraries, topics, brokers, and physical hosting nodes. It bridges code-level and architecture-level quality by synthesizing modular SCA metrics into a **Code Quality Penalty (CQP)**. It then applies a hierarchical **Reliability–Maintainability (RM)** attribution model grounded in the **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) standards (ISO/IEC, 2023a,b). Reliability decomposes into **Fault Tolerance** (FT, error propagation reach) and **Availability** (A, structural single-point-of-failure exposure), while **Maintainability** (M) quantifies coupling complexity and change ripple risk. The diagnostic pathway maps structural technical debt directly to stakeholder harm categories ($\mathbf{h}_{\text{QiU}}$) and routes explainable findings to specific engineering roles (Reliability Engineers, DevOps/SREs, and Software Architects). Over these metric vectors, it executes a catalog of nineteen formal publish–subscribe anti-patterns using adaptive box-plot thresholds. Section 9.1 shows composite criticality, betweenness, and degree centrality reaching comparable ranking quality, so the pathway's claim is not ranking accuracy but the deterministic, auditable decomposition a scalar centrality score cannot provide—a property of the design whose practical benefit to reviewers we do not evaluate here (Section 10.1).

- **The Counterfactual Prescriptive Pathway (Closed-Loop Refactoring Verification):** Rather than offering open-loop suggestions, the prescriptive engine compiles diagnostic findings into three typed graph-mutation operators: logical topic splitting, physical anti-affinity reallocation, and transport QoS contract hardening. It subjects every candidate refactoring to **counterfactual verification** on an in-memory sandbox graph (`MemoryRepository`), simulating failure cascades across multiple seeds and propagation thresholds. An edit is admitted if and only if its measured impact reduction exceeds the simulator's seed noise at every threshold ($\overline{\Delta I} > \kappa\sigma_{\text{seed}}$), and the composite policy is gated on a net positive System Risk Index delta ($\Delta\text{SRI} > 0$).

1.4 Contributions

This paper makes the following contributions to automated software quality assurance and code review:

1. **A Formal Diagnostic Pathway for Pub-Sub Software Quality Evaluation (Section 3):** We formalize a multi-level diagnostic model that ingests file-level SCA metrics via the Code Quality Penalty (CQP), derives multi-layer dependency projections (G_{analysis}), and computes a hierarchical Reliability–Maintainability (RM) attribution model grounded in ISO/IEC 25010 and ISO/IEC 25019 Quality-in-Use standards, including an explicit statement of the layer on which each dimension is well-posed.

2. **A Comprehensive Catalog of 19 Publish–Subscribe Anti-Patterns and Bad Smells (Section 4):** We formalize nineteen architectural anti-patterns and smells as topological signatures over a structural metric vector with adaptive box-plot thresholds, classify each pattern's finding scope (component vs. edge vs. system-level), and report which patterns actually trigger on the benchmark corpus.

3. **A Calibration and Validity Analysis of Deterministic Architectural Diagnosis (Section 9.1–9.4):** Rather than reporting ranking correlation alone, we measure agreement

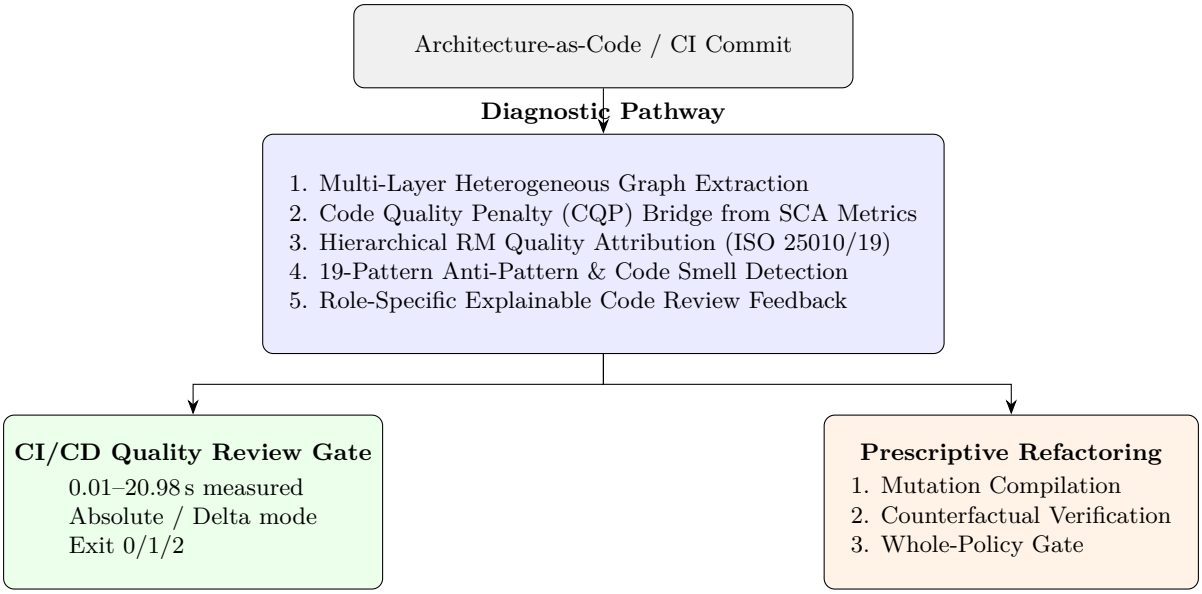


Figure 1: Architectural overview of SaG-Prescribe: the Diagnostic Pathway (Section 3–4) is the shared substrate feeding both the CI/CD quality gate (Section 7) and the counterfactual prescriptive engine (Sections 5–6).

(Cohen's κ), ranking-quality saturation across three predictors (NDCG@10, Top-k), the mechanism behind the catalog's over-flagging (edge- vs. component-scoped findings), and the structural degeneracy of the Availability dimension on the application-layer projection—evidence a purely accuracy-framed evaluation would omit.

4. **A Prescriptive Refactoring Pipeline with Counterfactual Verification (Sections 5–6):** We introduce a rule-based refactoring compiler translating diagnostic findings into three graph-mutation operators (topic splitting, anti-affinity reallocation, QoS hardening), verified against a discrete-event cascade simulator using a two-level margin-gating criterion. We disclose the exact automation footprint (5 of 19 patterns automated, 14 advisory).

5. **An Automated Code Review Bot and CI/CD Quality Gate (Section 7):** We operationalize the Diagnostic Pathway as a pre-merge quality gate featuring standardized exit codes (0/1/2), measured at 0.01–20.98 s across the corpus, and formalize delta-aware merge-base regression semantics. We are explicit that only absolute gating is implemented and that it blocks on all eight scenarios (Section 7.2); delta-aware gating, the mode that would make the gate usable, is specified but not yet implemented or evaluated.

6. **Empirical Evaluation and Methodological Findings (Sections 8–9):** Evaluated across eight benchmark scenarios (29 to 520 vertices; 12 to 300 applications), we report: (a) diagnostic ranking saturates across predictors, with no pair distinguishable under a paired test (composite $\rho = 0.485$ vs. degree 0.519, $p = 0.95$; both around NDCG@10 ≈ 0.82–0.86) and exhibits a Simpson's paradox pooling effect ($\rho = 0.028$ pooled vs. 0.503 on applications); (b) the catalog is a sensitive but chance-level-agreeing screen (recall = 0.942, precision = 0.253, Cohen's $\kappa = -0.002$, implicating 94.8% of components, 98.4% of findings not component-scoped); (c) the Availability dimension is degenerate at the application layer (SPOF $F_1 = 0.0$ in all 8 scenarios); (d) counterfactual verification admits 1128 of 1589 candidates (71.0%), with anti-affinity reallocation demonstrating 77.7% survival, and every scenario achieving statistically significant risk reduction (Wilcoxon $W = 0, p = 0.0156, n = 7$); (e) diagnostic review gating executes in 0.01–20.98 s over the measured

range, though the cost grows superlinearly with system size and is not characterised beyond it (Section 9.6).

1.5 Relationship to Prior Work

This submission relates to two prior bodies of work along different axes.

First, our earlier conference paper (Yigit and Buzluca, 2025) introduced the multi-layer pub-sub graph model this paper builds on and a two-term Criticality Score, $CS(v) = 0.7 \cdot C_B(v) + 0.3 \cdot AP(v)$ (betweenness and articulation-point membership), validated on a ten-application simulated system against Reachability Loss—the fraction of directed application-pair paths broken by removing v—reporting $\rho = 0.94$. That validation shares its substrate with its predictor: Reachability Loss is itself a connectivity statistic on the same undirected application graph that C_B and AP are computed on, and the benchmark is a single, small system. This paper re-measures the analogous claim under an independent discrete-event cascade oracle (I_{comp}, Section 4.5) across eight scenarios up to 300 applications, on the directed `DEPENDS_ON` projection rather than the undirected pub-sub graph, and finds a substantially lower correlation ($\rho = 0.485$) that is not surpassed by any single structural predictor (Section 9.1). The mechanism is identified in Section 9.4: on the projection this paper measures, the application layer has essentially no articulation points, so $AP(v)$—and hence 30% of the prior work's composite—carries almost no signal at this layer. Section 3.5 states the general form of this substrate-adequacy constraint before the results are presented.

Second, a companion manuscript by the same authors—*Software-as-a-Graph: A Static System Analysis Framework for Pre-Deployment Quality Gating and Failure Simulation of Publish-Subscribe Middleware*, currently under review at another journal and therefore mentioned here in text rather than listed as a reference—introduces the heterogeneous graph schema, simulation kernels, and a learned Graph Neural Network (GNN) for failure-impact prediction. The two submissions share the graph model, the scenario corpus, and the cascade simulator, and are otherwise disjoint: that manuscript contributes the learned predictive pathway, while this paper contributes the **Diagnostic Pathway** for automated code review, the Code Quality Penalty (CQP), the 19-pattern pub-sub anti-pattern catalog with its scope and coverage analysis, the prescriptive refactoring compiler, the counterfactual verification engine, the CI/CD quality gate, and the comprehensive empirical evaluation of detection, calibration, and prescription. No result, table, or figure is shared between the two. Detection figures previously reported from preliminary conference workshops are re-measured here with strict artifact reproducibility.

1.6 Organization

Section 2 surveys related work. Section 3 formalizes the Diagnostic Pathway and software quality model. Section 4 presents the anti-pattern catalog. Section 5 defines the closed-loop optimization objective. Section 6 details the prescriptive pipeline. Section 7 describes CI/CD integration. Sections 8 and 9 present the experimental design and results. Section 10 discusses implications and threats to validity, and Section 11 concludes.

2 Background and Related Work

2.1 Publish–Subscribe Middleware Dependability

Message-oriented middleware research has historically focused on protocol correctness, broker replication, network load balancing, and runtime QoS verification. Classical broker fault-tolerance techniques replicate or partition broker processes to withstand crash faults (Pallickara et al., 2007; Chang et al., 2014), while log-centric architectures such as Apache Kafka provide distributed commit-log durability (Wang et al., 2015). In DDS and ROS 2 ecosystems, recent

studies investigate the probabilistic latency characteristics of QoS-driven retransmission protocols (Lee et al., 2025b) and static verification of interdependent QoS policies (Lee et al., 2025a). While Chaos Engineering (Basiri et al., 2016) injects runtime faults into active deployments, it introduces operational hazards and occurs late in the software lifecycle. SaG-Prescribe shifts resilience analysis left into CI/CD, evaluating Architecture-as-Code descriptors statically before deployment.

2.2 Automated Code Review and Static System Analysis

Automated code review tools traditionally focus on lexical syntax, coding conventions, and localized security vulnerabilities via AST parsing and data-flow analysis (e.g., SonarQube, SpotBugs). Recent research has advanced automated code review using deep learning and large language models for review comment generation and code change assessment (Tufano et al., 2022; Lu et al., 2021). However, these approaches remain confined to source-file boundaries. They cannot infer system-wide topological cascades resulting from decentralized pub-sub message routing. SaG-Prescribe introduces *Static System Analysis (SSA)*, augmenting file-level code quality metrics with global architectural graph models to review structural and middleware-level design flaws.

2.3 Software Quality Models and Explainable Diagnostic Pathways

Software quality models, standardized in ISO/IEC 25010 (Product Quality) and ISO/IEC 25019 (Quality-in-Use), provide structured taxonomies decomposing quality into maintainability, reliability, and usability (ISO/IEC, 2023a,b). While these standards establish top-down quality characteristics, mapping them to concrete, automated measurements in distributed architectures remains challenging. Classical software analytics often collapse multiple structural metrics into a single opaque score or rely on black-box machine learning classifiers that lack explanatory transparency (Menzies and Rahman, 2018). In automated code review, developers reject opaque recommendations without clear causal rationale.

Two distinct notions of "explainability" appear in this literature, and they are not interchangeable. *Post-hoc attribution* methods (e.g., SHAP, LIME, and attention-weight inspection for GNN-based scorers) approximate why an already-trained black-box model produced a given score, but the explanation is itself an approximation, can be unstable across re-fits, and does not constrain the model to respect any declared quality taxonomy. *Score decomposition*, by contrast, is intrinsic: the score is defined as a declared weighted sum over named sub-characteristics, so the same computation that produces $Q(v)$ also produces its provenance. The Diagnostic Pathway in SaG-Prescribe adopts the latter: it decomposes structural and code metrics into explicit Reliability (Fault Tolerance, Availability) and Maintainability dimensions, mapping technical debt directly to stakeholder harm vectors and specific engineering roles, with no separate explanation model to validate or drift out of sync with the scorer it explains.

2.4 Anti-Pattern and Code Smell Catalogs

In object-oriented software engineering, Fowler's refactoring catalog (Fowler, 1999) and Brown et al.'s AntiPatterns taxonomy (Brown et al., 1998) formalized bad smells (e.g., God Class, Feature Envy) alongside remediation strategies. Suryanarayana et al. systematized design smells using formal structural metrics (Suryanarayana et al., 2014). In distributed systems, Richardson cataloged microservices design patterns (Richardson, 2018), while Taibi et al. established empirical taxonomies of microservices anti-patterns (e.g., Cyclic Dependency, Shared Database, Distributed Monolith) (Taibi et al., 2020). Our catalog (Section 4) adopts this proven specification structure—formal topological rule, affected quality dimension, and concrete

remediation—while addressing the unique structural anomalies of publish–subscribe middleware (e.g., broker saturation, topic fan-out explosion, QoS policy mismatches).

At the architectural level specifically, smell detection is already a tooled discipline. Mo et al. formalised hotspot patterns as detectable architecture smells and validated them against maintenance cost (Mo et al., 2015); Arcan (Fontana et al., 2017), Designite (Sharma et al., 2016) and DV8 (Cai and Kazman, 2019) operationalise overlapping catalogs, and Azadi et al. (Azadi et al., 2019) and Mumtaz et al. (Mumtaz et al., 2021) survey what these tools agree and disagree on. Several of their detectors—cyclic dependency, hub-like dependency, unstable dependency—correspond directly to `CYCLE`, `HUB_AND_SPOKE` and `UNSTABLE_INTERFACE` in our catalog. The distinction we claim is substrate, not concept: these tools operate on module, package or class dependency graphs recovered from source, and have no representation for a topic, a broker, or a QoS contract, which is where the pathologies specific to publish–subscribe middleware live. We do not, however, evaluate against them, and Section 11.2 records running one on the projected dependency graph as the external baseline this paper lacks. The architecture recovery and decay literature (Garcia et al., 2013; Le et al., 2015) supplies the extraction techniques such a comparison would need.

A second body of work bears directly on Section 9.2's calibration finding. Lenarduzzi et al. (Lenarduzzi et al., 2020) report that SonarQube's rule violations correlate weakly with the fault-proneness they are taken to indicate—a widely deployed rule-based screen whose flags do not discriminate the outcome they purport to predict. Our catalog's chance-level component-level agreement is, against that background, less an anomaly than an instance of a recurring pattern in rule-based quality screening, and one reason we frame it as a calibration result rather than a defect of any single threshold.

Software performance engineering maintains an older, parallel catalog. Smith and Williams (Smith and Williams, 2000, 2002) named recurring performance antipatterns and their remedies, and later work detects them automatically in Java applications (Trubiani et al., 2023), models microservice design patterns with queueing networks (Pinciroli et al., 2023), and reports architecture and performance anti-patterns co-occurring on a microservice benchmark (Avritzer et al., 2025). Their Blob and One-Lane Bridge resemble our `GOD_COMPONENT` and `BROKER_OVERLOAD`. The difference is the outcome: those patterns are judged by response time and resource use, whereas ours are compared against simulated failure impact, and we make no performance claim.

Scalability-fault research approaches a neighboring question from the code side. Scalability bugs can stay latent until a system reaches a large deployment (Leesatapornwongsa et al., 2017), and ScaleCheck exposes them through single-machine emulation (Stuardo et al., 2019). PCatch predicts, from small-scale runs, bugs that let a slowdown in non-scalable code propagate across jobs (Li et al., 2018). Zhu et al. (Zhu et al., 2026) derive eleven anti-patterns from 444 fault reports in ten distributed systems, Kafka among them, and find that iterative code whose iteration count grows with a scale dimension is not faulty in itself but becomes so in combination with a specific anti-pattern, a mechanism behind two-thirds of the faults they study. These techniques instrument or analyze implementation code that our model does not see, and we do not detect the code-level faults they target. Their combination finding has an architectural analogue in our `COMPOUND_RISK`, but we offer no evidence for that analogy, since the pattern does not trigger on our corpus (Table 3).

2.5 Refactoring Recommendation and Technical Debt Management

Automated refactoring recommendation has been investigated across multiple paradigms:

1. *Metric- and Rule-Based Recommenders:* Rule-driven engines identify code smells and suggest deterministic refactorings (Fowler, 1999; Suryanarayana et al., 2014).

2. *Search-Based Software Engineering (SBSE):* Multi-objective optimization algorithms (e.g., NSGA-II) search architectural trade-off spaces to optimize coupling and cohesion (Harman

et al., 2012; Aleti et al., 2013; Deb et al., 2002). Much SBSE work is open-loop with respect to resilience, optimising structural surrogates (coupling, cohesion, modularity) rather than a simulated failure outcome. This is not universal, and we narrow our claim accordingly: the Palladio/PerOpteryx line (Koziolek et al., 2011) places a quality-attribute model directly in the optimisation loop, evaluating candidate architectures against simulated performance, reliability and cost. What is distinctive here is therefore not simulation-in-the-loop as such, but the acceptance criterion—per-edit admission against discrete-event cascade semantics with an explicit seed-noise margin—and not the search, since SaG-Prescribe performs none (Section 5).

3. *Machine Learning-Based Refactoring:* Supervised and self-affirming models predict refactoring opportunities from historical commit data and code quality trajectories (AlOmar et al., 2021; Bavota et al., 2012; Ouni et al., 2017).

4. *LLM-Assisted Refactoring:* Recent foundation models assist in automated code transformation and refactoring explanation (White et al., 2024; Hou et al., 2024; Al-Kaswan et al., 2024).

SaG-Prescribe differs fundamentally in scope and verification: its scope is the global publish–subscribe topology, and its verification model is strictly closed-loop—every candidate refactoring is measured on a counterfactual sandbox graph against a cascade failure simulator before being presented to the architect.

2.6 Structural Criticality and Graph Centrality

Graph-theoretic analysis has long utilized centrality indices—betweenness (Freeman, 1977), PageRank, closeness, and articulation points—to identify critical vertices. Bakhtin et al. applied network centralities to microservice call graphs (Bakhtin et al., 2025), while complex network studies demonstrate scale-free, hub-dominated characteristics in software dependency graphs (Falci et al., 2018). Closest to this paper, our own earlier conference work (Yigit and Buzluca, 2025) combined betweenness and articulation-point membership into a two-term Criticality Score and validated it against reachability loss on a ten-application pub-sub system, reporting $\rho = 0.94$. Baldwin and Clark's design structure matrices (Baldwin and Clark, 2000) and combinatorial network reliability theory (Colbourn, 1987) provide formal foundations for topological fault tolerance. While the theoretical literature posits that multi-dimensional metrics should outperform simple centrality on heterogeneous graphs, our empirical findings (Section 9.1)—including a re-measurement of Yigit and Buzluca (2025)'s own claim under an independent cascade oracle and a larger, more heterogeneous benchmark corpus—show that scalar degree centrality remains a formidable baseline: it is not out-performed by the multi-dimensional composite on our corpus, and neither does it out-perform it, the paired difference being far from significant (Section 9.1). We note the limits of that comparison: both baselines are centralities computed within our own framework on our own projection, and no established external detector is evaluated (Section 11.2).

3 System Model and The Multi-Level Diagnostic Pathway

3.1 Heterogeneous Graph Formulation

A distributed publish–subscribe system is modeled as a typed, weighted, directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_E, w_V) \tag{1}$$

where the vertex type mapping $\tau_V : V \to T_V$ partitions components into five distinct semantic classes:

$$T_V = \{\text{Application}, \text{Library}, \text{Topic}, \text{Broker}, \text{Node}\} \tag{2}$$

- **Application ($V_{\mathbf{app}}$):** Active execution processes (e.g., ROS 2 nodes, microservices) that publish or subscribe to data channels.
- **Library ($V_{\mathbf{lib}}$):** Shared software libraries linked by applications.
- **Topic ($V_{\mathbf{topic}}$):** Named communication channels routing message streams.
- **Broker ($V_{\mathbf{broker}}$):** Message routing intermediaries (e.g., MQTT brokers, DDS participants).
- **Node ($V_{\mathbf{node}}$):** Physical or virtual compute hosts providing computational substrate.

The edge type mapping $\tau_E : E \to T_E$ assigns each link to an architectural relationship:

$$T_E = \{\text{PUBLISHES_TO}, \text{SUBSCRIBES_TO}, \text{ROUTES}, \\ \text{RUNS_ON}, \text{CONNECTS_TO}, \text{USES}\} \tag{3}$$

3.2 Derived Dependencies: The `DEPENDS_ON` Projection

To expose indirect failure propagation channels hidden behind asynchronous pub-sub decoupling, the Diagnostic Pathway derives explicit `DEPENDS_ON` relations (G_{analysis}) via formal projection rules (directed as dependent $\to$ dependency):

- **Application-to-Application:** Formed when application A_1 subscribes to a topic T published by application A_2: $(A_1 \to A_2) \in E_{\text{depends}}$.
- **Application-to-Broker:** Formed when application A relies on broker B routing topic T: $(A \to B) \in E_{\text{depends}}$.
- **Application-to-Library:** Formed when application A uses shared library L: $(A \to L) \in E_{\text{depends}}$, capturing simultaneous blast radius if L experiences a defect.
- **Broker-to-Broker / Node Colocation:** Captures shared-fate host vulnerabilities when multiple brokers or critical services share physical node N.

The projection is organized across four architectural layers:

1. **Application Layer (app):** $V_{\text{app}} \cup V_{\text{lib}}$, focusing on business logic dependencies.
2. **Infrastructure Layer (infra):** V_{node}, capturing compute host topology.
3. **Middleware Layer (mw):** $V_{\text{app}} \cup V_{\text{broker}}$, capturing message mediation.
4. **System Layer (system):** All five vertex types, capturing global cross-layer coupling.

3.3 The Code Quality Penalty (CQP): Bridging SCA and Architecture

To bridge source-code health with architectural risk during automated review, the Diagnostic Pathway ingests four file-level static code analysis (SCA) signals for Application and Library vertices: Lines of Code (`loc`, from `code_metrics.size.total_loc`), Weighted Methods per Class as a complexity proxy (`cyclomatic_complexity`, from `code_metrics.complexity.avg_wmc`), Lack of Cohesion in Methods (`lcom`, from `code_metrics.cohesion.avg_lcom`), and code-level instability, instability_code$(v) = C_e(v)/(C_a(v) + C_e(v))$, computed from afferent/efferent coupling (`avg_fanin`/`avg_fanout`). The SonarQube Technical Debt Ratio (`sqale_debt_ratio`) is ingested and persisted alongside these fields but does not enter the CQP formula below or any other scoring path (Table 1); it is exported for dashboard display only.

LOC, complexity, and LCOM are **min-max normalized independently within each node type**—Application and Library vertices form two separate populations, since their typical

scales differ—while instability is already bounded in $[0, 1]$ and is not renormalized. A population with zero variance (e.g., every node of a type carries no `code_metrics`) collapses to a normalized value of 0 for all its members rather than 1, so that "uniformly absent data" is not scored as "uniformly worst"; a population of exactly one measured node keeps its full normalized value of 1. These normalized signals synthesize into a per-component **Code Quality Penalty (CQP)**:

$$\begin{aligned}\mathrm{CQP}(v) = {} & 0.10\,\mathrm{loc_norm}(v) + 0.35\,\mathrm{complexity_norm}(v) \\ & + 0.30\,\mathrm{instability_code}(v) + 0.25\,\mathrm{lcom_norm}(v)\end{aligned} \tag{4}$$

$\mathrm{CQP}(v) \in [0, 1]$ provides an explicit, quantitative bridge from source-code technical debt to architectural criticality, entering directly into the Maintainability dimension of the RM model.

Table 1: CQP input fields: what feeds the score vs. what is ingested but does not.

Field	CQP term	Status
`loc`	$0.10 \times$ loc_norm	Scores
`cyclomatic_complexity` (avg WMC)	$0.35 \times$ complexity_norm	Scores
`lcom`	$0.25 \times$ lcom_norm	Scores
`avg_fanin/avg_fanout`	$0.30 \times$ instability_code	Scores
`sqale_debt_ratio`	—	Ingested, exported, never scored

3.4 Hierarchical Reliability–Maintainability (RM) Quality Attribution

Grounded in **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) (ISO/IEC, 2023a,b), the Diagnostic Pathway decomposes structural quality into two primary dimensions—**Reliability** and **Maintainability**—providing explainable, role-specific diagnostics (Figure 2, Table 2).

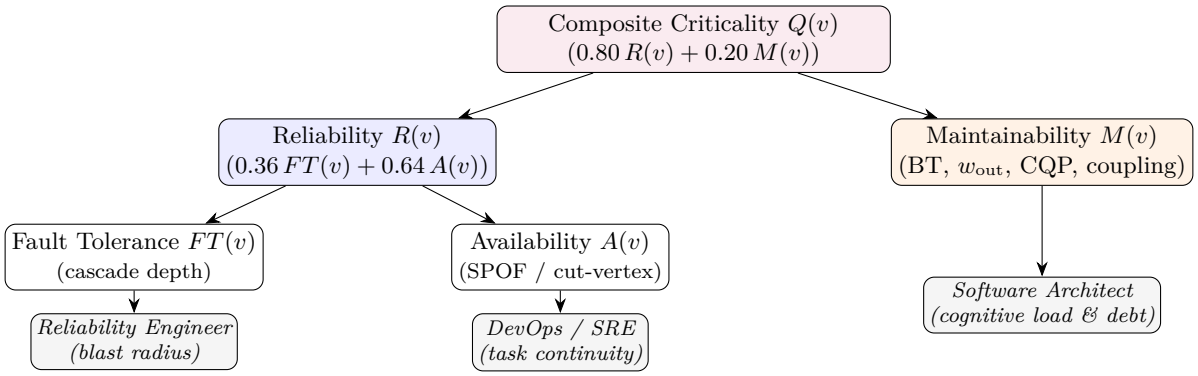


Figure 2: The hierarchical RM Diagnostic Attribution Model and stakeholder role mapping. Each arrow is a scoring dependency computed by the same deterministic pass, not a post-hoc explanation of a separately trained model.

Formal Quality Definitions.

- **Definition D1 (Component Criticality):** Let $G_l = (V_l, E_l)$ be a layer-projected graph. Component criticality $\mathrm{crit}_l : V_l \to [0, 1]^2 \times [0, 1]$ maps vertex v to metric vector $\mathbf{s}(v) = [R(v), M(v)]^T$ and composite score $Q(v)$, estimating Quality-in-Use degradation.
- **Definition D2 (Relationship Criticality):** Let $e = (u, v) \in E_l$ be a dependency edge. Relationship criticality $\mathrm{crit}_l(e) \in [0, 1]$ estimates Quality-in-Use loss resulting from link disruption.

Table 2: RM Quality Dimensions, Stakeholder Roles, and ISO/IEC 25019 Quality-in-Use Mapping.

Dim.	Sub-Characteristic	Engineering Role	High Score Meaning	ISO 25019 Harm
R	**Fault Tolerance** (FT)	Reliability Engineer	Cascades deeply; large blast radius	Indirect Harm
	Availability (A)	DevOps / SRE	Single point of failure; partition	Primary Task Loss
M	Structural Coupling	Software Architect	High change ripple; code-level debt	Maintainer Effort

- **Definition D3 (Consequence Metric):** Criticality measures structural consequence given failure. Failure likelihood must be supplied externally from operational MTTF telemetry.
- **Definition D4 (Relative Distribution):** Criticality classifications are relative to the score distribution of system S at layer l.

Structural Inputs. The RM formulas below consume the following structural quantities, computed on the layer-projected graph G_l. Let $\varepsilon = 10^{-9}$.

- $\mathrm{RPR}(v)$: PageRank on the transposed graph G_l^{T}, rank-normalised over the component population. `DEPENDS_ON` edges point dependent $\rightarrow$ dependency, so failure propagates *against* edge direction; reverse PageRank is therefore the cascade-reach estimator, not forward PageRank.
- $\mathrm{DG}_{\mathrm{in}}(v) = \deg^-(v)/(|V_l|-1)$ and $\mathrm{DG}_{\mathrm{out}}(v) = \deg^+(v)/(|V_l|-1)$.
- **Multi-Path Coupling Index:** $\mathrm{MPCI}(v) = \sum_{e\in\mathrm{In}(v)} \max(\mathrm{paths}(e)-1,\,0)\,/\,(|V_l|-1)$, where $\mathrm{paths}(e)$ counts the distinct pub-sub channels realising dependency e.
- **Fan-Out Criticality** (Topic vertices only): $\mathrm{FOC}(t) = \ln(1+f(t))\,s(t)\,/\,\max_{t'}[\ln(1+f(t'))\,s(t')]$, for topic publication frequency $f(t)$ in Hz and subscriber count $s(t)$.
- **Directed Articulation Score:** with $L(H)$ the order of the largest connected component of H, $\mathrm{AP}_c^{\mathrm{out}}(v) = 1 - L(G_l \setminus \{v\})/(|V_l|-1)$, $\mathrm{AP}_c^{\mathrm{in}}(v)$ the same on G_l^{T}, and $\mathrm{AP}_{c,\mathrm{directed}}(v) = \max(\mathrm{AP}_c^{\mathrm{out}}(v), \mathrm{AP}_c^{\mathrm{in}}(v))$. This is continuous, not the binary cut-vertex predicate.
- **Bridge Ratio:** $\mathrm{BR}(v) = |\{e \in \mathrm{bridges}(G_l^{\mathrm{und}}) : v \in e\}| / \deg^{\mathrm{und}}(v)$.
- **Connectivity Degradation Index:** $\mathrm{CDI}(v) = \min\big([\bar{d}(G_l \setminus \{v\}) - \bar{d}(G_l)]/\bar{d}(G_l),\ 1\big)$, for $\bar{d}$ the average shortest-path length over reachable pairs.
- $\mathrm{BT}(v)$: betweenness centrality; $\mathrm{CC}(v)$: clustering coefficient; $\mathrm{PC}(v)$: path complexity (count of distinct paths through v, normalised); $w(v)$: rank-normalised QoS component weight; $w_{\mathrm{in}}(v), w_{\mathrm{out}}(v)$: rank-normalised summed QoS weight over in- and out-edges.

Mathematical Formulation of Quality Dimensions.

1. **Fault Tolerance (FT).** Estimates cascade reach. For Application, Library, Broker and Node vertices, an enhanced cascade-depth potential CDPot discounts vertices that forward

more than they absorb, then amplifies by multi-path coupling:

$$\begin{aligned} \mathrm{CDPot}_{\mathrm{base}}(v) &= \frac{\mathrm{RPR}(v) + \mathrm{DG}_{\mathrm{in}}(v)}{2}\left(1 - \min\left(\frac{\deg^+(v)}{\max(\deg^-(v), \varepsilon)},\, 1\right)\right) \\ \mathrm{CDPot}(v) &= \min\big(\mathrm{CDPot}_{\mathrm{base}}(v)\,(1 + \mathrm{MPCI}(v)),\; 1\big) \\ FT(v) &= 0.45\,\mathrm{RPR}(v) + 0.30\,\mathrm{DG}_{\mathrm{in}}(v) + 0.25\,\mathrm{CDPot}(v) \end{aligned} \tag{5}$$

Topic vertices take a separate branch, since a topic's blast surface is its subscriber set moderated by publisher redundancy:

$$FT_{\mathrm{topic}}(t) = 0.50\,\mathrm{FOC}(t) + 0.50\,\mathrm{FOC}(t)(1 - \min(w_{\mathrm{in}}(t), 1)) \tag{6}$$

2. **Availability (A).** Evaluates single-point-of-failure exposure. The interaction term $\mathrm{QSPOF}(v) = \mathrm{AP}_{c,\mathrm{directed}}(v) \cdot w(v)$ deliberately reuses $\mathrm{AP}_{c,\mathrm{directed}}$ so that structural SPOF risk is not masked at low QoS weight; its effective weight therefore ranges over [0.25, 0.45] as $w(v)$ ranges over $[0, 1]$:

$$\begin{aligned} A(v) = {} & 0.25\,\mathrm{AP}_{c,\mathrm{directed}}(v) + 0.20\,\mathrm{QSPOF}(v) \\ & + 0.20\,\mathrm{BR}(v) + 0.25\,\mathrm{CDI}(v) + 0.10\,w(v) \end{aligned} \tag{7}$$

3. **Maintainability (M).** Blends betweenness, efferent QoS out-degree, the Code Quality Penalty, a path-complexity-enriched coupling risk, and inverse clustering. With $\iota(v) = \deg^+(v)/(\deg^-(v) + \deg^+(v) + \varepsilon)$ the deployment-graph instability,

$$\begin{aligned} \mathrm{CouplingRisk_enh}(v) &= \min\big(1,\; [1 - |2\iota(v) - 1|]\,(1 + 0.10\,\mathrm{PC}(v))\big) \\ M(v) &= 0.35\,\mathrm{BT}(v) + 0.30\,w_{\mathrm{out}}(v) + 0.15\,\mathrm{CQP}(v) \\ &\quad + 0.12\,\mathrm{CouplingRisk_enh}(v) + 0.08\,(1 - \mathrm{CC}(v)) \end{aligned} \tag{8}$$

4. **Reliability Composite (R).** Blends Fault Tolerance and Availability with $\alpha = 0.36$:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v) \tag{9}$$

Stakeholder Harm Projection Matrix. The diagnostic vector $\mathbf{s}_{\mathrm{RM}}(v) = [R(v), M(v)]^T$ projects into ISO/IEC 25019 Quality-in-Use stakeholder harm scores $[H_{\mathrm{Ben}}, H_{\mathrm{Risk}}, H_{\mathrm{Acc}}]^T$ via transformation matrix $\mathbf{M}_{\mathrm{RM}\to\mathrm{QiU}}$:

$$\mathbf{h}_{\mathrm{QiU}}(v) = \mathbf{M}_{\mathrm{RM}\to\mathrm{QiU}} \cdot \mathbf{s}_{\mathrm{RM}}(v) = \begin{bmatrix} 0.75 & 0.25 \\ 0.80 & 0.20 \\ 0.60 & 0.40 \end{bmatrix} \begin{bmatrix} R(v) \\ M(v) \end{bmatrix} \tag{10}$$

We are explicit about what this projection does and does not add. $\mathbf{M}_{\mathrm{RM}\to\mathrm{QiU}}$ is row-stochastic and maps a two-vector into three dimensions, so it cannot introduce information: each harm score is an affine function of the same (R, M) pair, and any domain-weighted scalarisation onto quality-in-use harm is algebraically identical to scoring the same RM vector under different composite weights ($\mathbf{M}^{\mathsf{T}}\boldsymbol{\omega}$). The three rows are close to collinear besides. $\mathbf{h}_{\mathrm{QiU}}$ is therefore a *re-presentation* of RM in stakeholder-facing terms—useful for routing a finding to the role that owns it, which is its purpose—and not an independent assessment along three axes. We do not report it as a quantity distinct from a reweighted $Q(v)$, and readers should not treat it as one.

Composite criticality blends the dimensions under declared weights $(w_R, w_M) = (0.80, 0.20)$:

$$Q(v) = 0.80\,R(v) + 0.20\,M(v) \tag{11}$$

Provenance of the Composite Weights. $\alpha = 0.36$ and $(w_R, w_M) = (0.80, 0.20)$ are not independently tuned constants, and we state their derivation rather than citing a method name. They are a re-parameterisation of a retired four-dimensional composite whose weights were elicited by Analytic Hierarchy Process (Saaty, 1980) over the criteria (Availability, Reliability, Maintainability, Vulnerability), yielding the priority vector $(0.43, 0.24, 0.17, 0.16)$. When the Vulnerability dimension was withdrawn from the model, the remaining three were renormalised over their residual mass 0.84, giving Availability 0.512, Reliability 0.286, Maintainability 0.202. The hierarchical form above reproduces exactly these: $w_R(1-\alpha) = 0.512$, $w_R\,\alpha = 0.288$, $w_M = 0.20$, each within 0.003 of the renormalised AHP value. Two consequences follow, and we state both. First, the weights should not be re-tuned independently of that derivation without re-eliciting the underlying comparison matrix. Second, and more importantly for a reader assessing this model, they inherit whatever subjectivity the original elicitation carried; we report no sensitivity analysis over α or (w_R, w_M) in this paper, and Section 11.2 lists it as required work. Section 9.4 gives a concrete reason it matters: on the application-layer projection, $A(v)$ is near-constant, so a large region of α-space produces near-identical rankings and the nominal 0.64 weight on Availability overstates its actual influence on that layer. Components are categorized into five severity tiers (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`, `MINIMAL`) using adaptive box-plot thresholding ($Q > Q_3 + 1.5\,\mathrm{IQR}$ for `CRITICAL`; $Q_3 < Q \leq$ upper fence for `HIGH`).

3.5 Substrate Adequacy: Which Layer Each Dimension Presumes

The RM model's dimensions are not equally well-posed on every layer projection of Section 3.2. $FT(v)$ and $M(v)$ are continuous functionals of reachability and coupling and vary smoothly across projections of any size. $A(v)$ is different: its dominant terms, $\mathrm{AP}_{c,\mathrm{directed}}$ (directed articulation-point membership) and BR (bridge ratio), are discrete structural predicates that fire only where the projected graph actually contains cut vertices or bridge edges. A projection that happens to be densely interconnected—many alternate paths between any two applications—can have *zero* such structures by construction, in which case $A(v)$ collapses to a near-constant value for every vertex in that projection, regardless of the underlying system's true availability risk.

This is a property of the projection, not a defect in the formula: $A(v)$ is answering "does removing v disconnect this graph," and on a well-connected projection the honest answer is "no, for almost every v." Because $A(v)$ carries $(1-\alpha) = 0.64$ of $R(v)$'s weight and $R(v)$ carries $w_R = 0.80$ of $Q(v)$'s weight, a near-constant $A(v)$ on a given layer flattens roughly half of the composite score on that layer specifically—not a general property of the RM model, but a property of scoring it on a projection where the availability substrate is degenerate. Section 9.4 reports this concretely for the application-layer projection used throughout Section 9.1's headline evaluation. The practical implication is that Availability-focused review (SPOF and `BROKER_OVERLOAD` detection) should be scored on a layer projection where cut structures are structurally possible—the middleware or system layer (Section 3.2), where brokers and physical nodes reintroduce genuine bottlenecks—rather than assumed to transfer unchanged from the application layer.

4 A Catalog of Architectural Anti-Patterns for Publish–Subscribe Systems

4.1 Anti-Patterns vs. Bad Smells in Automated Review

In automated code review, findings fall into two distinct confidence tiers:

- **Architectural Anti-Pattern:** A proven structural pathology known to cause systemic vulnerability or failure cascades, requiring structural intervention (e.g., Single Point of Failure, God Component, Broker Overload).

- **Architectural Bad Smell:** A localized structural heuristic signaling potential technical debt, warranting review and localized refactoring (e.g., Chatty Pair, QoS Mismatch, Orphaned Topic).

Publish–subscribe architectural anomalies leave distinct **topological signatures** in the dependency graph. Because these signatures are computable from static descriptors, automated review bots can flag them before code is deployed.

4.2 Detection Methodology

Detection operates over the same fourteen structural inputs that feed RM attribution (Section 3.4): Reverse PageRank (RPR), in-degree and out-degree (DG_{in}, DG_{out}), multi-path coupling index (MPCI), topic fan-out criticality (FOC), betweenness (BT), QoS-weighted in- and out-degree (w_{in}, Topic-only, w_{out}), clustering coefficient (CC), path complexity (PC), directed articulation score ($AP_{c,directed}$), bridge ratio (BR), connectivity degradation index (CDI), and component weight (w)—plus the synthesized Code Quality Penalty (CQP, Section 3.3), for fifteen scoring metrics in total, verified field-for-field against the `SCORING` role in the codebase's metric registry. Forward centralities (PageRank, Closeness, Eigenvector) feed only the learned GNN pathway of the companion manuscript described in Section 1.5 and play no role in this deterministic detector layer. Detectors employ **adaptive box-plot thresholds** ($Q_3 + k\,\mathrm{IQR}$, fixed at $k = 1.5$ in this work; Section 11.2 discusses sweeping k).

4.3 Catalog Overview

Table 3 summarizes the nineteen anti-patterns and smells spanning three severity tiers, four catalog categories, and four finding scopes—**Component** (attributed to one vertex), **Edge** (attributed to one directed relationship), **Pair** (attributed to a bidirectional component pair), and **System** (a corpus-wide statistic, not attributable to any single element). Only Component-scoped findings are attributable to a single vertex; Edge, Pair, and System findings are grouped as non-component-scoped throughout Section 9.2. The scope column matters for interpreting Section 9.2: component-level agreement metrics (precision, recall, Cohen's κ) are measured against component-scoped findings, while edge-scoped findings only implicate components indirectly, as endpoints.

4.4 Representative Pattern Walkthroughs

1. **SPOF (Single Point of Failure):** A node whose removal disconnects the graph ($AP_{c,directed}(v) > 0$). Detection combines graph connectivity testing with QoS weighting ($QSPOF(v) = AP_c(v) \times w(v)$). Remediation generates container anti-affinity constraints.

2. **GOD_COMPONENT:** An entity exhibiting extreme betweenness and CRITICAL maintainability ($BT(v) > 0.30 \wedge M(v) \in \text{CRITICAL}$). Remediation decomposes pub-sub channels via logical topic splitting.

3. **BROKER_OVERLOAD (Cautionary Benchmark Case):** A broker routing $\geq 2\times$ the median broker's traffic. *Cautionary finding:* In scenario S05 (two equally overloaded brokers), the within-population relative median rule cannot fire because each broker *is* the median. This highlights the necessity of absolute capacity-aware thresholds (Section 11.2).

4. **CHATTY_PAIR:** Two components maintaining bidirectional high-frequency dependencies across separate topics ($(u \to v) \wedge (v \to u)$), representing logical coupling masquerading as pub-sub decoupling.

Table 3: Publish–Subscribe Architectural Anti-Pattern and Bad Smell Catalog. **Trig.** marks patterns that fire at least once on the eight-scenario benchmark corpus (Section 4.6).

Pattern	Sev.	Category	Scope	Detection Signal	Autom. Refactoring	Trig.
SPOF	CRITICAL	Availability	Component	Articulation point $\wedge$ QoS SPOF score	Anti-affinity reallocation	✓
SYSTEMIC_RISK	CRITICAL	Reliability	System	Share of CRITICAL components $> 20\%$	Advisory	
CYCLE	HIGH	Architecture	System	Strongly connected component / loop	Advisory	✓
GOD_COMPONENT	CRITICAL	Maintainability	Component	Extreme betweenness $\wedge$ CRITICAL $M(v)$	Logical topic splitting	
BOTTLENECK_EDGE	HIGH	Availability	Edge	Edge betweenness centrality outlier	Logical topic splitting	✓
BROKER_OVERLOAD	HIGH	Availability	Component	Broker availability $\geq 2\times$ median load	Anti-affinity reallocation	
DEEP_PIPELINE	HIGH	Reliability	System	Path length $\geq \max(5, P_{75})$	Advisory	(excl.)
TOPIC_FANOUT	MEDIUM	Reliability	Component	Topic subscriber out-degree outlier	Logical topic splitting	
CHATTY_PAIR	MEDIUM	Maintainability	Pair	Bidirectional weight product $> \tau_{\text{chatty}}$	Advisory	
QOS_MISMATCH	MEDIUM	Reliability	Edge	Pub/Sub QoS-weight gap $> \tau_{\text{qos}}$	Advisory (Manual / Relay)	✓
ORPHANED_TOPIC	MEDIUM	Maintainability	Component	Zero in- or out-degree on graph	Advisory	
UNSTABLE_INTERFACE	MEDIUM	Maintainability	Component	High CouplingRisk_enh $\wedge$ high $M(v)$	Advisory	✓
BRIDGE_EDGE	HIGH	Availability	Edge	Graph-theoretic bridge cut-edge	Advisory	✓
FAILURE_HUB	CRITICAL	Reliability	Component	Reliability outlier $\wedge$ high out-degree	Logical topic splitting	
CONCENTRATION_RISK	MEDIUM	Reliability	System	Top-3 PageRank share > 0.50	Advisory	
HUB_AND_SPOKE	MEDIUM	Maintainability	Component	Low clustering $\wedge$ degree > 3	Logical topic splitting	✓
CHAIN	MEDIUM	Architecture	System	Degree-bounded linear subgraph	Advisory	
ISOLATED	MEDIUM	Architecture	Component	Zero total degree	Advisory	✓
COMPOUND_RISK	CRITICAL	Architecture	Component	Co-occurring SPOF + God/Hub finding	Anti-affinity reallocation	

5. **QOS_MISMATCH:** A publisher offering weaker reliability than the subscriber requires ($w_{\text{pub}}(u) < w_{\text{sub}}(v) - \tau$). In DDS/ROS 2, this produces silent connection dropouts at runtime without compile-time errors.

4.5 Detection Validation Methodology

Detection accuracy is validated against an independent discrete-event cascade simulation oracle ($I_{\text{comp}}(v)$) that models cascading message dropouts and queue exhaustion under component removal.

4.6 Catalog Coverage on the Benchmark Corpus

A catalog contribution should disclose which of its patterns are actually exercised by its own evaluation corpus, not only which are specified. Across all eight benchmark scenarios (Section 8.2), `DEEP_PIPELINE` is excluded for the reason given in Section 9.6, and of the remaining eighteen active detectors, exactly **eight trigger at least one finding** (Table 3, **Trig.** column): Bottleneck Dependency (`BOTTLENECK_EDGE`, 4974 findings across the corpus), QoS Policy Mismatch (`QOS_MISMATCH`, 1243), Unstable Interface (64), SPOF (17), Hub-and-Spoke (14), Dependency Cycle (`CYCLE`, 9), Bridge Edge (5), and Isolated Component (3). The remaining ten active patterns—including `GOD_COMPONENT`, `FAILURE_HUB`, `BROKER_OVERLOAD`, `TOPIC_FANOUT`, `CHATTY_PAIR`, `ORPHANED_TOPIC`, `CONCENTRATION_RISK`, `CHAIN`, `SYSTEMIC_RISK`, and `COMPOUND_RISK`—are specified and unit-tested but never fire on this corpus. This is consistent with, and explains, Section 7.1's automation-footprint disclosure being scoped to what the corpus can exercise rather than what the catalog specifies; it does not indicate the unexercised patterns are miscalibrated, since triggering depends on topological structure this synthetic corpus may simply not contain (e.g., `SYSTEMIC_RISK` requires more than 20% of a scenario's components at `CRITICAL`, which none of the eight scenarios reach).

Of the 6329 total findings the triggered eight produce, only **98 (1.5%)** are attributable to a single component (SPOF 17, Unstable Interface 64, Isolated Component 3, Hub-and-Spoke 14); the remaining **6231 (98.4%)** are Edge- or System-scoped (Bottleneck Dependency 4974 and QoS Policy Mismatch 1243 alone account for 6217 of these). This scope imbalance is the mechanism behind Section 9.2's implication-rate and agreement findings: a catalog whose active findings are overwhelmingly non-component-scoped will still implicate a large fraction of components, as edge endpoints, even when its direct component-level agreement with ground truth is weak.

5 Closed-Loop Optimization Objective

The prescriptive engine consumes the Diagnostic Pathway's output directly—severity tiers from Section 3.4 and pattern findings from Section 4—rather than re-deriving its own criticality signal, so diagnosis and prescription cannot silently drift apart. The idealised prescriptive objective is a graph transformation $\Delta(G)$ producing mutated topology $G' = \Delta(G)$ that minimises global failure impact subject to a budget constraint:

$$\min_{\Delta} \sum_{v \in V} I^*_{\Delta(G)}(v) \quad \text{subject to} \quad \text{Cost}(\Delta) \leq \mathcal{B} \tag{12}$$

We state Equation 12 to locate this work relative to search-based architecture optimisation (Harman et al., 2012; Aleti et al., 2013; Koziolek et al., 2011), and we are explicit that SaG-Prescribe does not solve it. No search over the space of transformations is performed, no cost model $\text{Cost}(\cdot)$ is instantiated, and no budget $\mathcal{B}$ is enforced anywhere in the implementation or the evaluation. What the system implements is a *screening rule* over a candidate set that the Diagnostic Pathway fixes in advance—a greedy, verification-gated approximation whose admitted set carries no optimality guarantee of any kind. Casting it as constrained minimisation

would invite a comparison with SBSE methods that this design could not survive, and would misdescribe what the results in Section 9.5 measure. Section 11.2 lists cost-aware subset selection as the step that would begin to close the gap.

Candidate refactorings are compiled from entities flagged `CRITICAL` or `HIGH` by the Diagnostic Pathway, and SaG-Prescribe applies a **two-level acceptance filter** to them:

1. **Per-Edit Counterfactual Acceptance Filter:** Each candidate edit δ is applied in isolation to a sandboxed copy of G and simulated across propagation thresholds $\Theta = \{0.1, 0.2, 0.5\}$ and seed set $S = \{42, 123, 456\}$. It is retained if and only if its mean impact reduction exceeds the simulator's seed noise at every threshold:

$$\overline{\Delta I}_\theta(\delta) > \kappa \cdot \sigma_{\text{seed},\theta}(\delta) \quad \forall \theta \in \Theta \tag{13}$$

   where $\kappa = 1.0$ is the noise-rejection margin.

2. **Whole-Policy Acceptance Gate:** The surviving subset of edits Δ_{admitted} is applied jointly to G, and the resulting system is re-evaluated. The whole policy is accepted if and only if the net System Risk Index improves:

$$\Delta\text{SRI} = \text{SRI}_{\text{baseline}} - \text{SRI}_{\text{mutated}} > 0 \tag{14}$$

6 The SaG-Prescribe Prescriptive Pipeline

6.1 Hexagonal Core Architecture

SaG-Prescribe implements a hexagonal ports-and-adapters architecture. The domain service depends on the `IGraphRepository` port:

- `Neo4jRepository`: Bolt-driven graph store for persistent, production deployments.
- `MemoryRepository`: High-speed, in-memory graph repository ensuring thread-safe, isolated sandboxing for counterfactual verification in CI/CD pipelines.

6.2 The Seven Pipeline Stages

1. **Stage 1 (Model Ingestion):** Ingest JSON/YAML Architecture-as-Code descriptors and file-level SCA metrics.
2. **Stage 2 (Topological Projection):** Construct multi-layer graph G_{analysis} and derived `DEPENDS_ON` edges.
3. **Stage 3 (Diagnostic Attribution):** Compute CQP, RM dimensions (FT, A, R, M), and ISO/IEC 25019 harm mapping.
4. **Stage 4 (Anti-Pattern Detection):** Run the 19-pattern detection engine with adaptive box-plot thresholds.
5. **Stage 5 (Candidate Compilation):** Compile candidate refactoring mutations from `CRITICAL`/`HIGH` findings.
6. **Stage 6 (Counterfactual Verification):** Execute two-level simulation verification in `MemoryRepository`.
7. **Stage 7 (Review Interface):** Emit verified refactoring blueprint and CI/CD quality gate verdicts.

6.3 Refactoring Mutation Operators

1. **Logical Topic Splitting (Operator 1):** Targets `TOPIC_FANOUT`, `GOD_COMPONENT`, and `FAILURE_HUB`. Decomposes a multi-publisher topic T into dedicated sub-topics $\{T_a : a \in P(T)\}$, isolating data flows and bounding blast radius.

2. **Physical Anti-Affinity Reallocation (Operator 2):** Targets `SPOF`, `BROKER_OVERLOAD`, and `COMPOUND_RISK`. Generates orchestration anti-affinity constraints to migrate co-located critical components from shared compute host N_{from} to isolated host N_{to}, updating `RUNS_ON` and `CONNECTS_TO` edges.

3. **Transport QoS Contract Hardening (Operator 3):** Fires on `CRITICAL`/`HIGH` channels operating under volatile contracts (`BEST_EFFORT` / `VOLATILE`), upgrading them to `RELIABLE` and `TRANSIENT_LOCAL`.

Automation Footprint Disclosure. Exactly **five of the nineteen patterns** directly trigger an automated mutation operator (`SPOF` → Operator 2; `GOD_COMPONENT`, `BOTTLENECK_EDGE`, `FAILURE_HUB`, `HUB_AND_SPOKE` → Operator 1). Operator 3 triggers from generic criticality tiers. The remaining fourteen patterns provide advisory recommendations for human review. Of these five, two—`SPOF` and `BOTTLENECK_EDGE`—are among the eight patterns Section 4.6 finds actually trigger on the benchmark corpus; the other three (`GOD_COMPONENT`, `FAILURE_HUB`, `HUB_AND_SPOKE`) are specified operator triggers this corpus does not exercise. In practice, Rule 2 (anti-affinity reallocation) fires from RM criticality tier membership and `SPOF` findings rather than the catalog name directly, while Rule 1 (topic splitting) fires on multi-publisher/multi-subscriber congestion, RM-critical topics, or a topic feeding a `Bottleneck`/`Hub`-matched finding—so the corpus's candidate mix is shaped as much by criticality-tier membership as by which of the five triggering patterns fires; Section 9.5 reports the resulting per-operator breakdown.

6.4 Closed-Loop Counterfactual Verification Procedure

Candidate generation and per-edit verification are decoupled at the level that matters for the first filter: candidates are generated from G_{analysis} and diagnostic RM attribution, while each edit is simulated in isolation on the raw structural graph $G_{\text{structural}}$ within `MemoryRepository`, so the per-edit acceptance statistic $\overline{\Delta I}$ comes from an oracle that shares no computational substrate with the RM score that proposed the edit.

This independence does *not* extend to the second-level gate. ΔSRI is computed from the same RM dimensions used to compile candidates (Section 8.3 gives the identity $\text{SRI} = 0.5\,\bar{R}_w + 0.5\,\bar{M}_w$), so the whole-policy check is an internal-consistency test rather than an independent one. Section 9.5 accordingly reports the primary prescriptive outcome in cascade-impact units and treats ΔSRI as secondary; Section 10.5 states the residual circularity as a threat.

7 DevOps Integration and CI/CD Quality Gating

7.1 Automated Code Review Bot Architecture

To govern architectural quality during continuous evolution, the Diagnostic Pathway is packaged as a lightweight pre-merge review bot. When a pull request modifies system topology or QoS configurations, the bot analyzes the change, executes the 18 active anti-pattern detectors (Section 9.6 excludes `DEEP_PIPELINE`), attributes RM debt, and posts structured review comments.

7.2 Regression Semantics: Absolute vs. Delta-Aware Gating

- **Absolute Gating:** Evaluates the entire topology against global thresholds. However, real-world industrial systems carry intentional, risk-accepted legacy debt; blocking builds on pre-existing debt paralyzes development.
- **Delta-Aware Gating (Design):** Compares the PR candidate topology against the target branch merge-base, blocking only *newly introduced* anti-patterns or worsening severities, while honoring a signed **waiver register**. Absolute mode is implemented today; delta-aware mode is specified here for production integration but not yet implemented.

Absolute Gating Outcome on the Benchmark Corpus. Every one of the eight benchmark scenarios contains at least one `CRITICAL`- or `HIGH`-severity finding (Section 9.2), so under absolute gating the review bot returns **Exit Code 2 (Block) on all eight scenarios**. Given the 94.8% mean component-implication rate this catalog configuration produces (Section 9.1), this is the expected, mechanical consequence of absolute thresholds applied globally, not a scenario-specific anomaly—and it is precisely why delta-aware gating is a necessity for adoption rather than a convenience: an absolute gate that blocks every commit on legacy debt trains developers to ignore or bypass it.

7.3 Three-Tier Exit-Code Protocol

- **Exit Code 0 (Pass):** No findings above threshold; deployment permitted.
- **Exit Code 1 (Warning):** `MEDIUM`-severity smells detected; build passes with advisory comments.
- **Exit Code 2 (Block):** `CRITICAL` or `HIGH` anti-patterns detected; build fails, blocking merge.

8 Experimental Design

8.1 Research Questions

- **RQ1 (Diagnostic Ranking Efficacy):** How does the RM composite's rank correlation with cascade failure impact compare to single-metric structural baselines (betweenness, degree)?
- **RQ2 (Catalog Screening and Calibration):** How well does the anti-pattern catalog's component-level agreement match its precision/recall profile, and what accounts for its implication rate?
- **RQ3 (Attribution Validity and Substrate Adequacy):** Does the Availability dimension carry genuine signal on the layer this evaluation measures, how does stratification by node type affect the pooled ranking result, and how does this re-measurement compare to our prior published claim?
- **RQ4 (Prescriptive Refactoring Efficacy & Operator Survival):** Does the closed-loop prescriptive engine achieve statistically significant reductions in System Risk Index across heterogeneous scenarios, and how do individual refactoring operators survive counterfactual verification?
- **RQ5 (Verification Yield, Compositionality & CI/CD Feasibility):** What proportion of generated candidates survive verification, do individually admitted edits compose beneficially, and what is the execution latency of diagnostic gating and prescriptive verification across system scales?

8.2 Benchmark Scenarios

We evaluate eight benchmark scenarios spanning six architectural domains—autonomous vehicles, IoT smart cities, financial trading, healthcare, hub-and-spoke microservices, and hyper-scale enterprise systems—plus a minimal regression fixture (Table 4). Scenarios S01–S08 evaluate detection; S01–S07 evaluate prescription (S08 excluded as a minimal regression smoke test, not as a scale-driven exclusion). We flag an inconsistency this creates rather than leave it implicit: S08 is judged too trivial to prescribe on, yet enters every unweighted detection mean on equal footing with S07, which is twenty-five times its size. Section 9.1 shows this is not cosmetic—S08 alone determines the sign of the composite-versus-degree comparison—so detection means are reported there with and without it, and weighted by component count.

Table 4: Benchmark Scenario Scales and Topological Dimensions.

Scenario	Applications	Libraries	Topics	Brokers	Nodes	Total Edges ($\|E\|$)
S01 Autonomous Vehicle	80	20	40	4	8	797
S02 IoT Smart City	200	10	80	6	30	1322
S03 Financial Trading	60	18	35	5	6	580
S04 Healthcare	50	12	25	3	8	400
S05 Hub-and-Spoke	70	25	30	2	12	797
S06 Microservices Mesh	90	30	45	6	15	680
S07 Hyper-Scale Enterprise	300	50	120	10	40	3245
S08 Tiny Regression	12	4	8	2	3	101

8.3 Experimental Protocol and Metrics

- **Detection Metrics:** Spearman rank correlation $\rho(Q, I_{\text{comp}})$ (pooled and stratified), Cohen's κ, NDCG@10, Top-k overlap, precision, recall, and F_1 against simulation ground truth I_{comp} (Section 4.5). Baselines: betweenness and degree centrality. For $Q(v)$, betweenness, and degree, the predicted and true critical sets are both sized by the same box-plot-or-top-20% rule (Section 9.1), which pins precision, recall, and F_1 to near-equal values by construction; the catalog's flagged set has no such constraint, so its precision and recall are measured independently.

- **Prescriptive Metrics — primary:** mean fractional cascade-impact reduction $\overline{\Delta I}_{\text{policy}} = \text{mean}_v\,[I(v) - I'(v)]/I(v)$, where I and I' are per-component composite impact under the discrete-event cascade simulator on the baseline and fully mutated graphs. This is measured in the same independent-oracle units as I_{comp} above, and is the outcome we treat as primary. It is restricted to remediated components whose identity survives the mutation (reallocations and QoS upgrades); topic splitting replaces the original topic identifier and therefore has no stable before/after counterpart, so this measure does not cover that operator.

- **Prescriptive Metrics — secondary:** the System Risk Index, $\text{SRI} = 0.5(1-H_R)+0.5(1-H_M)$, where $H_d = 1 - \left(\sum_v w(v)\, d(v)\right)/\sum_v w(v)$ is the QoS-weighted health of dimension d. Substituting, $\text{SRI} = 0.5\,\bar{R}_w + 0.5\,\bar{M}_w$: the SRI is the QoS-weighted mean of the same R and M that constitute $Q(v)$, differing only in dimension weights (0.5/0.5 rather than 0.80/0.20). Because candidates are compiled from RM severity tiers, $\Delta\text{SRI} = \text{SRI}_{\text{baseline}} - \text{SRI}_{\text{mutated}}$ scores the prescription in the same units that selected it. We report it as an internal-consistency check, not as evidence of resilience gain, and Section 9.5 contrasts the two measures directly.

- **Yield Metrics:** per-operator candidate generation counts, admitted edit counts, and per-edit survival rates.

- **Verification Parameters:** $\kappa = 1.0$, $\Theta = \{0.1, 0.2, 0.5\}$, $S = \{42, 123, 456\}$. Section 9.6 reports the sensitivity of admission to κ.
- **Seed Protocol:** Detection is deterministic given a topology; the only stochastic element is the cascade oracle. Detection results are therefore reported at the canonical seed 42, and stability is established by a separate five-seed sweep ($\{42, 123, 456, 789, 2024\}$) reported in Section 9.3.
- **Two Oracles, Not One:** This paper's ground truth, $I_{\text{comp}}(v)$, is an independent discrete-event cascade simulator modeling message dropouts and queue exhaustion under component removal on the directed `DEPENDS_ON` projection. Our prior work (Yigit and Buzluca, 2025) validated against a different oracle, Reachability Loss, on the undirected pub-sub graph—a distinction Section 9.4 returns to when reconciling the two papers' correlation figures.

9 Empirical Results

9.1 Diagnostic Ranking Efficacy (RQ1)

Table 5 presents detection validation at the application layer across all eight scenarios; Table 6 compares the three ranking predictors across four independent quality measures.

Table 5: Diagnostic detection validation (app layer) against simulation ground truth $I_{\text{comp}}(v)$. The F_1 column is reported for continuity with prior work but carries little information: predicted and true critical sets are sized by the same rule, which pins precision, recall and F_1 to equal values for $Q(v)$ by construction (Section 8.3). Catalog P and R are measured on an unconstrained flagged set and are not subject to that artifact

Scenario	n	$\rho(Q, I)$	F_1	Top-5	Catalog P	Catalog R	Implicated %	Gate (s)
S01 Autonomous Vehicle	80	0.457	0.600	0.40	0.244	0.950	97.5%	0.59
S02 IoT Smart City	200	0.718	0.780	0.40	0.310	0.880	71.0%	0.56
S03 Financial Trading	60	0.365	0.667	1.00	0.237	0.933	98.3%	0.40
S04 Healthcare	50	0.451	0.538	0.60	0.213	0.769	94.0%	0.16
S05 Hub-and-Spoke	70	0.466	0.611	0.80	0.257	1.000	100.0%	0.92
S06 Microservices Mesh	90	0.405	0.565	0.40	0.261	1.000	97.8%	0.35
S07 Hyper-Scale Enterprise	300	0.480	0.627	0.20	0.251	1.000	99.7%	20.98
S08 Tiny Regression	12	0.538	0.333	0.60	0.250	1.000	100.0%	0.01
Mean	—	**0.485**	**0.590**	**0.550**	**0.253**	**0.942**	**94.8%**	**3.00**

Table 6: Predictor Comparison, App Layer, Mean over 8 Scenarios. No predictor dominates on every measure.

Predictor	ρ	Cohen's κ	NDCG@10	Top-10
$Q(v)$ composite (RM)	0.4850	0.4514	0.8295	0.5875
Betweenness	0.4346	0.4139	**0.8592**	**0.6250**
Degree	**0.5187**	**0.4792**	0.8214	0.5625

Key Diagnostic Findings.

1. **No Predictor Is Distinguishable from Any Other.** Table 6's means cluster tightly—degree leads on ρ (0.519) and κ (0.479), betweenness on NDCG@10 (0.859) and Top-10 overlap (0.625), and the RM composite is never last by more than 0.03—but the means are the wrong statistic here, and we resist the temptation to read a ranking off them. Paired

across the eight scenarios, no difference approaches significance: Q versus degree gives Wilcoxon $p = 0.945$ (paired t, $p = 0.465$), and Q versus betweenness $p = 0.148$. Head to head, Q attains the higher ρ in **five of eight** scenarios. Degree's mean advantage rests almost entirely on `tiny_system`, the 12-component regression fixture, where $\rho_Q = 0.538$ against $\rho_{\text{deg}} = 0.848$—and where Q's own correlation is not significantly different from zero ($p = 0.071$, 95% CI $[-0.039, 0.852]$). Excluding that fixture the ordering reverses ($Q = 0.4774$, degree $= 0.4716$), and weighting each scenario by its component count rather than equally makes them indistinguishable (0.5151 versus 0.5170). The defensible conclusion is saturation, not a winner in either direction: on this corpus a multi-dimensional composite buys no ranking advantage over degree centrality, and degree buys none over the composite. What the RM composite offers instead is the decomposition Section 3.4 defines: a scalar centrality has no Fault Tolerance/Availability/Maintainability breakdown to route to a Reliability Engineer, an SRE, or an Architect. Whether that decomposition helps a reviewer in practice is a question about reviewers, and this evaluation does not answer it (Section 10.1).

2. **No Evidence of Degradation at Scale (Descriptive).** Stratifying by scenario size, mean ρ is 0.599 for the two large scenarios (≥ 150 components: S02, S07) versus 0.447 for the six smaller ones. We report this descriptively and draw no trend from it: the "large" group has $n = 2$, and the gap is driven entirely by S02 ($\rho = 0.718$)—S07, the largest system in the corpus at 300 applications, scores 0.480, essentially the corpus mean. The defensible reading is that ranking quality does not visibly collapse as these scenarios grow, not that it improves.

3. **Catalog Screening Behavior.** The anti-pattern catalog functions as a sensitive screen: mean recall is 0.942, precision is 0.253, implicating a mean of 94.8% of components at `CRITICAL/HIGH` severity. Section 9.2 explains the mechanism. Note that for the $Q(v)$/betweenness/degree predictors, precision, recall, and F_1 are numerically identical (Section 8.3)—an artifact of the equal-sized-set validation rule, not evidence that these predictors have no false positives or false negatives individually.

9.2 Catalog Screening and Calibration (RQ2)

Precision and recall do not merely understate how the catalog's findings relate to ground-truth criticality on this corpus—they are close to uninformative, and we set them aside before interpreting anything else. Two constructions pin them. First, the ground-truth critical set is the box-plot top tier ($\geq Q_3$), so it is the top quartile of components by construction: n_{critical}/n lies in $[0.250, 0.260]$ in all eight scenarios. Second, the catalog flags between 71.0% and 100% of components (five of eight scenarios exceed 97%; three flag every component). A screen that flags nearly everything has precision equal to the base rate and recall equal to one, arithmetically. That is what we observe: $|\text{precision} - \text{base rate}| \leq 0.06$ in every scenario, and exactly 0.000 in three of them. The mean precision of 0.253 reported in Table 5 is therefore a restatement of the corpus's 25% critical base rate, not a measurement of the catalog, and the mean recall of 0.942 is a restatement of its implication rate. Only `iot_smart_city` (flagging 71%, precision +0.060 above base rate) departs from this materially.

Cohen's κ is the one member of the triple that is not pinned, because it corrects for exactly the chance agreement the other two absorb. It measures chance-corrected agreement between the catalog's flagged/unflagged partition and the true critical/non-critical partition; it is $\kappa = \mathbf{-0.0022}$ at the application layer (mean over 8 scenarios, range -0.125 to $+0.141$) and $\kappa = \mathbf{-0.0300}$ at the system layer—both indistinguishable from chance agreement, despite recall of 0.942.

The mechanism is scope, not miscalibration of any single threshold: Section 4.6 shows that of the 6329 findings the eight triggered patterns produce across all scenarios, only 98

(1.5%) are attributable to a single component, while 6231 (98.4%) are Edge- or System-scoped. `n_flagged_directly`—findings whose entity is a component—is zero in 4 of 8 scenarios. A component is "implicated" almost entirely by being an endpoint of a flagged edge (chiefly `BOTTLENECK_EDGE` and `QOS_MISMATCH`), not by a component-level detector firing on it. High recall together with chance-level κ is therefore consistent, not contradictory: the catalog reliably flags *some* edge touching almost every genuinely critical component (hence high recall), while its component-level partition of flagged vs. unflagged carries almost no discriminative information beyond what a random partition of the same size would (hence $\kappa \approx 0$). We report this as a calibration finding: the catalog, as configured with a fixed $k = 1.5$ IQR fence (Section 4.2), is an indiscriminate component-level screen on synthetic topologies—Section 9.3 shows it behaves markedly better on transcribed real-world ones— and its practical CI/CD value depends on the delta-aware gating design of Section 7.2, not on tightening k alone—since tightening k would improve κ at recall's expense on the same edge-dominated finding mix.

9.3 Generalisation Check: Transcribed Real-World Topologies (RQ2, external validity)

The calibration result above is a property of the corpus as much as of the catalog, and the distinction is testable. We ran the same detection pipeline over three architectures transcribed from public open-source systems rather than generated: **Autoware.universe** (ROS 2/DDS autonomous driving; 32 applications, 24 topics with explicit DDS QoS profiles, 3 middleware brokers, 6 ECUs, 10 shared C++ libraries; 75 components), the **Train-Ticket** microservices benchmark (41 applications, 30 topics, 3 brokers, 8 nodes, 8 libraries; 90 components), and a **cloud microservice mesh** modelled on a production e-commerce reference architecture (22 applications, 20 topics, 4 brokers, 6 nodes, 8 libraries; 60 components). All three are converted to the typed multigraph of Section 3.1 by a documented adapter. Both corpora were swept over five oracle seeds ($\{42, 123, 456, 789, 2024\}$) at the system layer, so the comparison in Table 7 is like-for-like.

Table 7: Catalog behaviour on generated versus transcribed topologies, system layer, five oracle seeds. Cells are mean ± standard deviation over scenario-seed pairs. The catalog discriminates on transcribed architectures and does not on generated ones

Measure	**Generated** ($n = 40$)	**Transcribed** ($n = 15$)
Cohen's κ	-0.036 ± 0.049	$\mathbf{+0.296 \pm 0.100}$
Precision	0.237 ± 0.014	$\mathbf{0.402 \pm 0.060}$
Recall	0.887 ± 0.059	0.861 ± 0.105
F_1	0.374 ± 0.022	$\mathbf{0.545 \pm 0.061}$
Components implicated	$93.7\% \pm 3.2$	$\mathbf{56.3\% \pm 9.8}$

The contrast is substantial and it runs in the direction that matters. On transcribed architectures the catalog's chance-corrected agreement rises from indistinguishable-from-chance to $\kappa = +0.296$—fair agreement—precision rises by two thirds at essentially unchanged recall, and the implication rate falls from 93.7% to 56.3%. Per scenario, the directly-flagged component rate is 21.3% (Autoware), 18.3% (cloud mesh) and 14.4% (Train-Ticket), against a generated-corpus mean above 90%. The honest reading of Section 9.2 is therefore narrower than it first appears: *the catalog is an indiscriminate screen on our generated corpus*, and the indiscriminacy is substantially a property of how those topologies are constructed—plausibly their uniform density, which leaves few components that are not an endpoint of some outlier edge.

Two cautions keep this from being a stronger claim than it is. First, these are hand-transcribed *architectural models* of open-source systems, not automatically harvested code, so they inherit the transcriber's judgement about what to include; they are a step toward the industrial validation

Section 10.5 calls for, not that validation. Second, the ground-truth oracle is still the same simulator, so this tests generalisation of the corpus, not of the oracle.

Two results do not change. Detection is deterministic: findings counts have zero variance across all five seeds in every scenario of both corpora, so the only seed-driven variation in Table 7 enters through the oracle. And the review gate returns **Exit Code 2 (Block) on all eleven scenarios at every seed**, transcribed included—which strengthens rather than softens Section 7.2's argument: absolute gating is unusable on real architectures too, not merely on synthetic ones where the implication rate is inflated.

9.4 Attribution Validity and Substrate Adequacy (RQ3)

The Availability Dimension Is Degenerate at This Layer. Section 3.5 anticipates that $A(v)$ requires a projection where cut vertices and bridge edges actually occur. At the application layer, they largely do not: the `SPOF` detector scores $F_1 = 0.0$ in **all 8 scenarios**, predicting zero articulation points in 6 of 8 (the two exceptions, S02 and S04, predict 1–4 apiece against ground truth that is itself near-zero). Since $A(v)$ carries 0.64 of $R(v)$'s weight and $R(v)$ carries 0.80 of $Q(v)$'s weight, this means roughly half of the composite score is measured on a substrate with almost no structural variance at this layer—a concrete instance of the mechanism Section 3.5 states as a design-time caveat, not a failure discovered post hoc. This plausibly contributes to Table 6's saturation result: a composite devoting half its weight to a near-constant term should be expected to under-perform a predictor with no such dead weight.

Reconciling with Our Prior Published Result. Section 1.5 previewed the delta with (Yigit and Buzluca, 2025)'s $\rho = 0.94$; the mechanism above completes the explanation. That prior result used (a) Reachability Loss as ground truth, a connectivity statistic computed on the same undirected application graph its $CS(v)$ predictor is computed on—predictor and oracle share a substrate, which structurally favors high correlation; (b) a single ten-application system, where the specific mix of articulation points present is a property of that one topology, not a population statistic; and (c) $AP(v)$ contributing 30% of the composite by design, which is informative precisely because that topology had articulation points to find. This paper's I_{comp} oracle is an independent discrete-event cascade simulator sharing no computational substrate with the RM score, is measured across eight topologies up to 300 applications, and is measured on a directed `DEPENDS_ON` projection where—as shown above—articulation structure is largely absent. Both results correctly measure what they measured, but they do not measure the same quantity: ranking correlation against a connectivity-derived oracle is not a portable measure of a criticality score's diagnostic value.

Simpson's Paradox in Node Pooling. At the system layer (pooling all five node types), the pooled correlation is $\rho = 0.028$, while per-type strata show robust positive correlations (Application 0.503, Broker 0.395, Node 0.142)—the pooled figure falls *outside* the per-type range entirely (`pooled_inside_per_type_range: false`). This is Simpson's paradox from between-type score offsets: node types with systematically different score levels but internally consistent rankings pool into a near-zero aggregate correlation. Stratified evaluation is therefore a requirement rather than a presentational choice for heterogeneous software graphs, and every correlation reported outside this paragraph is single-type (Application) or explicitly stratified.

9.5 Prescriptive Refactoring Efficacy and Operator Survival (RQ4)

Table 8 presents prescriptive refactoring results under per-edit counterfactual verification across all seven benchmark scenarios; Table 9 breaks this down by refactoring operator. Baseline System Risk Index values span 0.3223 (S05) to 0.3749 (S02), and the corresponding mutated

values 0.3108 to 0.3288; the per-scenario ΔSRI column is the difference. We report the two outcome measures side by side rather than choosing between them silently, because they disagree.

Table 8: Prescriptive refactoring performance under per-edit counterfactual verification. $\overline{\Delta I}_{\text{policy}}$ is the mean fractional cascade-impact reduction on the fully mutated graph, measured by the independent discrete-event simulator (primary outcome); ΔSRI is the change in the framework's own RM-derived risk index (secondary, and not independent of the criterion that selected the edits — see Section 8.3). Baseline and mutated SRI values are given in the text. The two measures disagree in sign on S03 and S06

Scenario	$\overline{\Delta I}_{\text{policy}}$ (primary, oracle)	ΔSRI (secondary, RM)	**Cand.**	**Adm.**	**Rej.**	**Adm. %**
S01 Autonomous Vehicle	+0.0161	+0.0086	156	45	111	28.8%
S02 IoT Smart City	+0.0822	**+0.0461**	391	297	94	76.0%
S03 Financial Trading	**−0.0689**	+0.0108	125	22	103	17.6%
S04 Healthcare	**+0.1894**	+0.0109	99	36	63	36.4%
S05 Hub-and-Spoke	+0.0236	+0.0115	127	107	20	84.3%
S06 Microservices Mesh	**−0.0015**	+0.0097	163	142	21	87.1%
S07 Hyper-Scale Enterprise	+0.0060	**+0.0171**	528	479	49	90.7%
Mean / Total	**+0.0352**	—	**1589**	**1128**	**461**	**71.0%**
Scenarios improved	**5 / 7**	**7 / 7**				

Table 9: Candidate Generation vs. Verified Admission by Refactoring Operator.

Refactoring Operator	**Candidates**	**Admitted**	**Survival Rate**	**Primary Structural Target**
Physical Anti-Affinity Reallocation	1188	**923**	**77.7%**	Single points of failure, co-location
Transport QoS Contract Hardening	69	**40**	**58.0%**	Volatile transport contracts on topics
Logical Topic Splitting	332	**165**	**49.7%**	High-fan-out message channels, hubs
Total	**1589**	**1128**	**71.0%**	—

Key Prescriptive Findings.

1. **The Primary Result Is Positive but Not Significant.** Under the independent cascade oracle, the admitted policy reduces cascade impact in **five of seven** scenarios, with a mean reduction of 3.5% (median 1.6%; range −6.9% to +18.9%). A two-sided Wilcoxon signed-rank test against zero gives $p = 0.219$ ($n = 7$): the effect does not reach significance at this sample size. Two scenarios regress—S03 Financial Trading (−6.9%) materially, S06 Microservices Mesh (−0.15%) negligibly.

2. **The Framework's Own Index Disagrees, and We Report Why.** Scored in ΔSRI, all seven scenarios improve and a Wilcoxon test returns $W = 0$, $p = 0.0156$. It would be easy to present that as the headline, and we do not, for two reasons. First, SRI $= 0.5\,\bar{R}_w + 0.5\,\bar{M}_w$ is a reweighting of the same RM dimensions from which the candidate edits were compiled, so the test grades the prescription against the criterion that selected it. Second, Section 9.1 has already established that $Q(v)$—built from those same dimensions—ranks no better than degree centrality against this oracle, and Section 9.4 that half its weight sits on a near-constant term at this layer. A significance result in RM units is therefore an internal-consistency check and nothing stronger. Where the two measures disagree in sign (S03, S06), we take the oracle's verdict as the informative one.

3. **Substantial Yield:** Across 1589 generated candidates, **1128 edits (**71.0%**)** survive per-edit counterfactual verification.

4. **Anti-Affinity Reallocation Shows the Highest Survival Rate (**77.7%**)**, reaching near-100% in dense topologies (408/409 in Enterprise, 123/123 in Microservices). This is mechanistically plausible—Rule 2 fires only on nodes already flagged `SPOF` or RM-critical (Section 6.3), pre-filtering its pool—but survival rate measures how many edits clear the noise margin, not how much good they do: S07, whose admission rate is the highest in the corpus at 90.7%, delivers one of the smallest oracle-measured reductions (+0.6%). Transport QoS hardening survives at 58.0% and topic splitting at 49.7%.

5. **Coverage Caveat.** $\overline{\Delta I}_{\text{policy}}$ is measured over remediated components whose identity survives mutation, so it covers reallocation and QoS hardening but not topic splitting, which replaces the topic identifier (Section 8.3). The primary measure therefore under-covers the operator with the lowest survival rate, and closing that gap requires an identity-preserving formulation of the split operator (Section 11.2).

9.6 Verification Yield, Compositionality & CI/CD Feasibility (RQ5)

1. **Informative Rejection:** All 461 rejected edits record the binding propagation threshold θ where impact reduction failed to clear $\kappa\sigma_{\text{seed}}$, providing transparent review feedback.

2. **The Noise Margin Does Little Work.** Sweeping the acceptance multiple over $\kappa \in \{0, 0.5, 1, 1.5, 2, 3\}$ moves admission only from roughly 79% to roughly 62%; at the shipped $\kappa = 1.0$ it is 71.0%. Against $\kappa = 0$—a plain sign test asking only that the mean reduction be positive—the noise margin withholds about eight further percentage points of candidates. We report this because the two-level filter is presented in Section 5 as this paper's central methodological safeguard, and the sweep shows most of its selectivity comes from requiring a positive mean at *every* threshold rather than from the σ bar itself. The reason is that the cascade simulator is highly reproducible across seeds, so σ_{seed} is small and the median accepted edit clears it by a factor of 11.2 (inter-quartile range 4.0–33.5; only 6.9% of accepted threshold-statistics fall below 2σ). A margin calibrated against simulator determinism is not the same thing as a margin calibrated against real-world variability, and only the former is demonstrated here.

3. **Effect Magnitudes Are Small and Their Practical Significance Is Unestablished.** The median admitted edit reduces composite cascade impact by 4.2×10^{-5} at its binding threshold (inter-quartile range 1.0×10^{-5} to 1.2×10^{-4}; maximum 3.0×10^{-3}), against a median of -2.1×10^{-5} among rejected edits. The filter therefore separates sign reliably, which is what it is designed to do. Whether a reduction of this magnitude is operationally meaningful—whether it corresponds to any difference an operator would notice—is a question about the mapping from simulated impact to field outcomes that this paper does not address, and we make no claim about it.

4. **Compositional Dynamics:** S03 and S06 are the two scenarios where individually admitted edits compose to a net *increase* in cascade impact (−6.9% and −0.15% respectively, Table 8), despite every constituent edit having cleared the per-edit margin in isolation. Edits verified one at a time can therefore interact sub-additively, which is the strongest evidence in this paper for the necessity of a second-level whole-policy gate—and simultaneously evidence that the gate as currently formulated does not catch it, since both scenarios pass the $\Delta\text{SRI} > 0$ check. A whole-policy gate scored in oracle units rather than RM units would have rejected both.

5. **Diagnostic Review Gate Latency:** The complete diagnostic pipeline (graph construction, CQP computation, RM attribution, and 18 active anti-pattern detectors) executes in **0.01 s to 20.98 s** (Table 5). The eighteen detectors run in under **0.2 s total**; computational overhead is dominated by base metric computation.

6. `DEEP_PIPELINE` **Bounding:** The unconstrained `DEEP_PIPELINE` detector attempts exhaustive path enumeration, generating 247,761 findings on tiny fixtures and hanging on larger graphs. Excluding this detector (Section 4.6) restores sub-second gate execution.

7. **Prescriptive Verification Latency:** The full 7-scenario prescriptive sweep (14,301 simulation sweeps) completed in **2 h 47 min** using multi-core process parallelism (20 workers), establishing its viability for nightly or on-demand CI/CD refactoring runs.

10 Discussion and Threats to Validity

10.1 Diagnostic Explainability vs. Threshold Calibration

The Diagnostic Pathway provides deterministic, role-specific quality attribution (Reliability Engineer vs. SRE vs. Architect) by construction, and Section 9.1 shows that superior ranking is not where any advantage over a scalar centrality score could lie. We should be precise about the status of that claim. What we have established is a *property of the design*: the decomposition is intrinsic, auditable, and cannot drift out of sync with the scorer, because it is the scorer. What we have *not* established is that this property helps a practitioner—no user study, task-based evaluation, or expert assessment is reported here, and the argument that developers reject opaque recommendations (Section 2.3) is borrowed from the literature rather than measured on this tool. Readers should treat role-attributed diagnosis as a designed affordance whose practical benefit remains to be demonstrated (Section 11.2), not as a validated result of this paper. That value is not free, however: Section 9.2 shows the catalog's fixed $k = 1.5$ IQR fence produces near-chance component-level agreement ($\kappa \approx 0$) despite high recall, because 98.4% of active findings are edge-scoped and implicate components only as endpoints. Tightening k would trade recall for precision on the same finding mix, but would not by itself change the underlying scope imbalance—a catalog dominated by two edge-level detectors (`BOTTLENECK_EDGE`, `QOS_MISMATCH`) will always implicate components indirectly at scale. The mitigation this paper actually validates is architectural rather than purely statistical: absolute gating blocks on every one of the eight benchmark scenarios (Section 7.2), which is unusable in practice, while delta-aware gating—blocking only newly introduced findings relative to a merge-base—bounds the review surface to what a given change actually introduces, regardless of the base rate. Threshold recalibration (Section 11.2) and delta-aware gating are therefore complementary, not substitutes: one narrows what fires, the other narrows what is enforced.

10.2 The Verify-Before-Recommend Discipline

Counterfactual verification filters aggressively: 461 of 1589 candidate mutations (29.0%) failed to clear the noise margin $\kappa\sigma_{\text{seed}}$ at some propagation threshold and were withheld. We are careful about what this does and does not establish. Failing $\overline{\Delta I} > \kappa\sigma_{\text{seed}}$ means the measured improvement did not separate from simulator seed noise at $\kappa = 1.0$; the rejected set therefore mixes genuinely harmful edits, neutral edits, and edits that were beneficial but too noisy to certify at this margin. We cannot presently decompose it, because this evaluation contains no unverified control arm: we do not measure what would have happened had all 1589 candidates been applied, nor how a size-matched random edit set would compare. The value of the verification step is therefore argued mechanistically here and left to be measured directly (Section 11.2); it should not be read off the 29.0% rejection rate. What the discipline does buy, concretely, is that the reallocation result is a measurement rather than an assumption. The mechanistic argument for

why anti-affinity reallocation should work—removing SPOF co-location—is exactly the confident-sounding reasoning an open-loop recommender would ship regardless of its truth. Because every candidate is simulated independently, 77.7% is an observed survival rate. But survival is not benefit: Section 9.5 shows S07 admitting 90.7% of its candidates and returning one of the smallest oracle-measured gains in the corpus, and two scenarios composing individually-verified edits into a net regression. Per-edit verification bounds the damage a single bad edit can do; it does not establish that the surviving set is collectively worth applying.

10.3 Construct Validity: Substrate Adequacy

Beyond the general dependence on a discrete-event cascade simulator, this paper surfaces a more specific construct-validity threat: a composite score's sub-dimensions can be individually well- or ill-posed on a given graph projection, and scoring on an ill-posed projection silently degrades the composite without changing its formula. Section 3.5 states this as a design-time property; Section 9.4 measures its concrete instance—the Availability dimension carrying almost no variance on the application-layer projection used throughout this paper's headline detection evaluation, because that projection has essentially no articulation points to find. The mitigation is not a different formula for $A(v)$, but scoring Availability-sensitive review on a projection where cut structures can occur (the middleware or system layer), and reporting per-layer results rather than treating the application layer as universally representative. Any weighted sum over structurally heterogeneous sub-scores inherits this risk whenever one of its terms is a discrete graph predicate, so other RM-style composites on typed dependency graphs should check for it explicitly rather than only fixing it locally.

10.4 Comparison with Prior Self-Reported Results

Section 9.4 reconciles this paper's $\rho = 0.485$ against our own prior published $\rho = 0.94$ (Yigit and Buzluca, 2025). In discussion terms, the general lesson is broader than this one paper pair: a criticality score whose ground-truth oracle is itself a connectivity statistic on the same graph the score is computed from will tend to correlate well with that oracle almost by construction, and that correlation does not necessarily transfer to an oracle with independent causal mechanics (here, a discrete-event message-dropout and queue-exhaustion simulator). Identifying the mechanism, rather than merely disclosing the discrepancy, is what makes the result actionable for other researchers evaluating structural criticality scores against connectivity-derived oracles.

10.5 Further Threats to Validity

- **Construct Validity — Circularity of the Whole-Policy Gate:** The per-edit acceptance filter uses an oracle independent of the RM score, but the second-level gate does not: ΔSRI is a reweighting of the same R and M that compiled the candidates (Section 8.3). This is why Section 9.5 reports the oracle-measured reduction as primary, and it is a design defect rather than only a reporting one—a gate scored in RM units admitted the two policies the oracle scores as regressions.

- **Construct Validity — Shared Modelling Assumptions:** A stronger version of the threat Section 10.4 raises against our own prior work applies, in weaker form, to this evaluation. The scenario generator, the analyser, and the cascade simulator are all authored by us and share a model of how pub-sub failures propagate. I_{comp} shares no *computational* substrate with $Q(v)$, which is the property that matters most, but it does not share *no* assumptions: both are downstream of one generator's view of topology, QoS, and coupling. Replication on harvested industrial systems, with an oracle derived from observed incidents rather than simulation, is the only thing that settles this.

- **Construct Validity — Unmeasured Value of Verification:** No unverified control arm exists (Section 10.2), so the counterfactual filter's contribution is argued, not measured.
- **Construct Validity — Simulator Dependence:** Both detection and verification utilize a discrete-event cascade simulator. We mitigate simulator dependency by grounding operators in established distributed systems dependability patterns (anti-affinity scheduling, topic isolation, QoS hardening), and by validating the composite against SPOF ground truth directly (Section 9.4) rather than relying on ranking correlation alone.
- **Internal Validity:** Decoupled in-memory repositories ensure no circular data leakage between diagnostic candidate generation and counterfactual simulation.
- **External Validity:** The eight scenarios carrying this paper's headline results are synthetic domain models. Section 9.3 partially addresses this with three architectures transcribed from public open-source systems (Autoware.universe, Train-Ticket, a cloud microservice mesh), on which the catalog discriminates considerably better ($\kappa = +0.296$ against -0.036 on generated topologies). That result cuts two ways and we report both: it is evidence the catalog is not inherently indiscriminate, and it is evidence our generated corpus is unrepresentative in a way that shaped Section 9.2's headline. It remains a partial mitigation—these are transcribed architectural models, not automatically harvested codebases, and they are scored against the same simulator—so validation on harvested industrial systems with an incident-derived oracle remains high-priority future work.
- **Conclusion Validity:** Both Wilcoxon tests in Section 9.5 have $n = 7$, the minimum at which a two-sided signed-rank test can reach $p < 0.05$ at all; the $p = 0.0156$ obtained in RM units is the best attainable value at this sample size, not evidence of a large effect. The primary, oracle-measured test does not reach significance ($p = 0.219$). More fundamentally, the seven scenarios are configurations of a single generator rather than a sample from any defined population, so the inferential frame a signed-rank test presupposes is not really available. Per-scenario effect sizes with dispersion, as reported in Table 8, carry more information here than either p-value.
- **Internal Validity — Uncharacterised Cost Growth:** Gate latency rises from 0.56 s at 200 applications to 20.98 s at 300 (Section 9.6). No complexity analysis is given for the dominant stage, and the CI/CD feasibility claim should not be extrapolated beyond the measured range.

11 Conclusion and Future Work

11.1 Conclusion

This paper presented **SaG-Prescribe**, bringing the **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation for distributed publish–subscribe systems. It integrates code-level SCA metrics (CQP), multi-layer heterogeneous dependency modeling, hierarchical RM quality attribution informed by ISO 25010/25019, and formal anti-pattern detection with counterfactually verified refactoring. We describe this as a mechanism for *addressing* the Architecture-Code Gap rather than closing it: the code-level bridge itself, CQP, carries roughly 3% of the composite and is not ablated here, so the paper's motivating gap is bridged in design and not yet in evidence. Rather than claiming diagnostic superiority in ranking accuracy, the paper offers the diagnostic contribution as *attribution and calibration evidence*: three structural predictors saturate to the point of being statistically indistinguishable (Section 9.1), the anti-pattern catalog is an indiscriminate component-level screen on our generated corpus whose mechanism we identify—though it discriminates far better ($\kappa = +0.296$)

on three architectures transcribed from open-source systems, which locates the negative result in the corpus rather than the catalog (Sections 9.2–9.3), and the Availability dimension's substrate adequacy is stated as a modeling property and measured directly rather than assumed (Section 9.4)—including an explicit, mechanistically explained re-measurement of our own prior published criticality-ranking claim. On the prescriptive side, counterfactual verification admits 71.0% of refactoring candidates and reduces simulator-measured cascade impact in five of seven scenarios (mean +3.5%), an effect we report as promising but not statistically significant at this sample size ($p = 0.219$); the framework's own risk index improves in all seven, but scores the prescription in the units that selected it and we decline to lead with it. Diagnostic review gating executes in 0.01–20.98 s over the measured range, with cost growth that is superlinear and not yet characterised beyond it.

11.2 Future Work

1. **Corpus Realism as a First-Class Variable:** Section 9.3 shows the catalog's headline calibration result is largely a property of the generated corpus. Characterise which generator parameters drive the 94% implication rate—density is the leading candidate—and either calibrate the generator against transcribed topologies or move the headline evaluation onto them.

2. **An Unverified Control Arm:** Re-run the seven scenarios applying (a) all 1589 candidates, (b) a size-matched random subset, and (c) the 1128 admitted edits, comparing final oracle-measured impact. This is the experiment that would convert the verify-before-recommend discipline from a mechanistic argument into a measured result, and it requires no new infrastructure.

3. **Oracle-Scored Whole-Policy Gate:** Replace the $\Delta\text{SRI} > 0$ acceptance check with one scored in cascade-impact units, breaking the circularity Section 10.5 identifies. Both scenarios that regress under the oracle pass the current gate.

4. **Identity-Preserving Topic Split:** Reformulate the split operator so the resulting sub-topics retain a traceable link to the original identifier, bringing the lowest-survival operator inside the coverage of the primary outcome measure.

5. **CQP Ablation:** Measure $\rho(Q, I_{\text{comp}})$ and severity-tier assignment with and without the Code Quality Penalty. CQP carries $0.15 \times 0.20 = 3\%$ of the composite, and the Architecture-Code Gap this paper opens with is currently closed by an untested term.

6. **Weight Sensitivity Analysis:** Sweep α and (w_R, w_M) and report how ranking and tier assignment move, as Section 3.4's provenance paragraph notes is outstanding.

7. **Practitioner Evaluation of Role Attribution:** A controlled study comparing triage with and without the RM decomposition and role routing, measured on time-to-diagnosis and diagnostic accuracy. This is what the attribution claim (Section 10.1) requires and does not yet have.

8. **External Tool Baseline:** Compare detection against an established architectural smell detector (Fontana et al., 2017; Sharma et al., 2016) on the projected dependency graph, rather than only against centralities computed inside this framework.

9. **Detection Threshold Recalibration:** Sweep the box-plot fence multiplier k in $Q_3 + k\,\text{IQR}$—already a parameter of `BoxPlotClassifier`, but hardcoded at $k = 1.5$ in the anti-pattern detectors (Section 4.2)—against the precision/κ/recall trade-off Section 9.2 measures, rather than assuming the classical outlier convention is the right operating point for a CI/CD screen.

10. **Layer-Aware Availability Scoring:** Score $A(v)$ on a projection where cut structures are structurally possible (Section 3.5), and report Reliability/Availability results per-layer by default rather than defaulting to the application layer alone.
11. **Path-Bounded `DEEP_PIPELINE`:** Restrict path enumeration to bound execution time.
12. **Delta-Aware CI/CD Gating:** Implement merge-base diffing and waiver registers.
13. **Combinatorial Subset Verification:** Explore greedy forward-selection to optimize multi-edit composition.
14. **Dynamic κ Estimation:** Adapt noise margins based on empirical simulator variance.
15. **Per-Candidate Latency Profiling:** Instrument fine-grained verification timing.
16. **Operator Attribution Ablation:** Isolate per-operator ΔSRI contributions.
17. **Catalog Extension for Hybrid Architectures:** Incorporate REST/gRPC hybrid microservice patterns.
18. **Industrial Telemetry Replication:** Validate against live ROS 2 and DDS production traces, including a direct replication of (Yigit and Buzluca, 2025)'s reachability-loss validation on the larger corpus used here.
19. **LLM-Assisted Pull-Request Synthesis:** Integrate generative code models to automatically draft pull requests implementing verified refactoring blueprints.

Declarations

Funding: This work was supported in part by the Scientific and Technological Research Council of Türkiye (TÜBİTAK), grant number [GRANT-NUMBER].

Competing Interests: The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to the work submitted for publication.

Related Submissions and Overlap Disclosure: A companion manuscript by the same authors, *Software-as-a-Graph: A Static System Analysis Framework for Pre-Deployment Quality Gating and Failure Simulation of Publish-Subscribe Middleware*, is currently under review at another journal. The two submissions share three research artifacts—the heterogeneous graph schema, the benchmark scenario corpus, and the discrete-event cascade simulator—and are otherwise disjoint. That manuscript contributes the learned Graph Neural Network pathway for failure-impact prediction and its evaluation; this manuscript contributes the deterministic Diagnostic Pathway, the Code Quality Penalty, the nineteen-pattern anti-pattern catalog, the prescriptive refactoring compiler, the counterfactual verification engine, and the CI/CD quality gate. No result, table, or figure appears in both. We disclose this so the editor can assess the overlap directly.

Data Availability: All scenario topologies, generator configurations, result artifacts, seeds, and execution logs supporting the findings of this study are available in the replication package archived at [REPOSITORY-URL], persistent identifier `[TODO(DOI): mint a Zenodo DOI before submission]`. The two entry points that regenerate this paper's tables are:

```
PYTHONPATH=. python reproduce/detection_validation.py --layer app \
    --output results/detection_validation_app.json
PYTHONPATH=. python reproduce/run_prescribe_all.py --kappa 1.0
```

The first regenerates Tables 5 and 6; the second regenerates Tables 8 and 9. The prescriptive sweep takes approximately 2 h 47 min on 20 cores.

Author Contributions: [To be completed via the submission interface, per the journal's Submitting Declarations requirements.]

Ethics Approval: Not applicable.

References

Ali Moeen Al-Kaswan, Toufique Ahmed, Maliheh Izadi, Anand Ashok Sawant, Premkumar Devanbu, and Arie van Deursen. Automatic refactoring using large language models: A study on extract method and rename. *Empirical Software Engineering*, 29(4):98, 2024.

Aldeida Aleti, Barbora Buhnova, Lars Grunske, Anne Koziolek, and Indika Meedeniya. Software architecture optimization methods: A systematic literature review. *IEEE Transactions on Software Engineering*, 39(5):658–683, 2013.

Eman AlOmar, Mohamed Wiem Mkaouer, and Ali Ouni. Can refactoring be self-affirmed? an exploratory study on how developers document their refactorings. *Empirical Software Engineering*, 26(6):121, 2021.

Alberto Avritzer, Andrea Janes, Catia Trubiani, Helena Rodrigues, Yuanfang Cai, Daniel Sadoc Menasché, and Álvaro José Abreu de Oliveira. Architecture and performance anti-patterns correlation in microservice architectures. In *Proc. IEEE 22nd International Conference on Software Architecture (ICSA)*, pages 60–71, 2025. doi: 10.1109/ICSA65012.2025.00016.

Umberto Azadi, Francesca Arcelli Fontana, and Davide Taibi. Architectural smells detected by tools: A catalogue proposal. In *Proc. IEEE/ACM International Conference on Technical Debt (TechDebt)*, pages 88–97, 2019. doi: 10.1109/TechDebt.2019.00027.

Alexey Bakhtin, Marcelo Esposito, Valentina Lenarduzzi, and Davide Taibi. Network centrality as a new perspective on microservice architecture. In *Proc. IEEE International Conference on Software Architecture (ICSA)*, pages 72–83, 2025.

Carliss Y. Baldwin and Kim B. Clark. *Design Rules, Volume 1: The Power of Modularity*. MIT Press, 2000.

Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016. doi: 10.1109/MS.2016.60.

Gabriele Bavota, Bernardino De Carluccio, Rocco Oliveto, Massimiliano Di Penta, and Andrea De Lucia. When does a refactoring induce bugs? an empirical study. In *Proc. IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 104–113, 2012.

William H. Brown, Raphael C. Malveau, Hays W. McCormick, and Thomas J. Mowbray. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. John Wiley & Sons, 1998.

Yuanfang Cai and Rick Kazman. Dv8: Automated architecture analysis tool suites. In *Proc. IEEE/ACM International Conference on Technical Debt (TechDebt)*, pages 53–54, 2019. doi: 10.1109/TechDebt.2019.00015.

Ting Chang, Sisi Duan, Hein Meling, Sean Peisert, and Haibin Zhang. P2s: A fault-tolerant publish/subscribe infrastructure. In *Proc. 8th ACM International Conference on Distributed Event-Based Systems (DEBS)*, pages 198–207, 2014.

Charles J. Colbourn. *The Combinatorics of Network Reliability*. Oxford University Press, 1987.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2): 182–197, 2002.

Daniel H. M. Falci, Omar A. Gomes, and Fernando S. Parreiras. Complex networks analysis for software architecture: An hibernate call graph study. *IEEE Access*, 6:62145–62155, 2018.

Francesca Arcelli Fontana, Ilaria Pigazzini, Riccardo Roveda, Damian Tamburri, Marco Zanoni, and Elisabetta Di Nitto. Arcan: A tool for architectural smells detection. In *Proc. IEEE International Conference on Software Architecture Workshops (ICSAW)*, pages 282–285, 2017. doi: 10.1109/ICSAW.2017.16.

Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.

Linton C. Freeman. A set of measures of centrality based on betweenness. *Sociometry*, 40(1): 35–41, 1977.

Joshua Garcia, Igor Ivkovic, and Nenad Medvidovic. A comparative analysis of software architecture recovery techniques. In *Proc. 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 486–496, 2013. doi: 10.1109/ASE.2013.6693106.

Mark Harman, S. Afshin Mansouri, and Yuanyuan Zhang. Search-based software engineering: Trends, techniques and applications. *ACM Computing Surveys*, 45(1):1–61, 2012.

Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(5):1–68, 2024.

ISO/IEC. Iso/iec 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model. Technical report, International Organization for Standardization, 2023a.

ISO/IEC. Iso/iec 25019:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model. Technical report, International Organization for Standardization, 2023b.

Anne Koziolek, Heiko Koziolek, and Ralf Reussner. Peropteryx: Automated application of tactics in multi-objective software architecture optimization. In *Proc. Joint ACM SIGSOFT Conference on Quality of Software Architectures and ACM SIGSOFT Symposium on Architecting Critical Systems (QoSA-ISARCS)*, pages 33–42, 2011. doi: 10.1145/2000259.2000267.

Duc Minh Le, Pooyan Behnamghader, Joshua Garcia, Daniel Link, Arman Shahbazian, and Nenad Medvidovic. An empirical study of architectural change in open-source software systems. In *Proc. 12th Working Conference on Mining Software Repositories (MSR)*, pages 235–245, 2015. doi: 10.1109/MSR.2015.29.

Sanghoon Lee, Junsoo Kang, and Kyung-Joon Park. Dependency chain analysis of ros 2 dds qos policies: From lifecycle tutorial to static verification. *IEEE Internet of Things Journal*, 12(4): 3890–3901, 2025a.

Sanghoon Lee, Hyun-Seok Park, Jiseok Chae, and Kyung-Joon Park. Probabilistic latency analysis of the data distribution service in ros 2. *IEEE Transactions on Industrial Informatics*, 21(3):2415–2426, 2025b.

Tanakorn Leesatapornwongsa, Cesar A. Stuardo, Riza O. Suminto, Huan Ke, Jeffrey F. Lukman, and Haryadi S. Gunawi. Scalability bugs: When 100-node testing is not enough. In *Proc. 16th Workshop on Hot Topics in Operating Systems (HotOS)*, pages 24–29, 2017. doi: 10.1145/3102980.3102985.

Valentina Lenarduzzi, Francesco Lomio, Heikki Huttunen, and Davide Taibi. Are SonarQube rules inducing bugs? In *Proc. IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 501–511, 2020. doi: 10.1109/SANER48275.2020.9054821.

Jiaxin Li, Yuxi Chen, Haopeng Liu, Shan Lu, Yiming Zhang, Haryadi S. Gunawi, Xiaohui Gu, Xicheng Lu, and Dongsheng Li. PCatch: Automatically detecting performance cascading bugs in cloud systems. In *Proc. 13th European Conference on Computer Systems (EuroSys)*, pages 1–14, 2018. doi: 10.1145/3190508.3190552.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Daxin Jiang, Ming Zhou, and Nan Duan. Codexglue: A machine learning benchmark dataset for code understanding and generation. In *Proc. NeurIPS Datasets and Benchmarks*, 2021.

Tim Menzies and Foyzur Rahman. Bad smells in software analytics papers. *IEEE Software*, 35 (4):100–103, 2018.

Ran Mo, Yuanfang Cai, Rick Kazman, and Lu Xiao. Hotspot patterns: The formal definition and automatic detection of architecture smells. In *Proc. 12th Working IEEE/IFIP Conference on Software Architecture (WICSA)*, pages 51–60, 2015. doi: 10.1109/WICSA.2015.12.

Haris Mumtaz, Paramvir Singh, and Kelly Blincoe. A systematic mapping study on architectural smells detection. *Journal of Systems and Software*, 173:110885, 2021. doi: 10.1016/j.jss.2020.110885.

Ali Ouni, Marouane Kessentini, Houari Sahraoui, and Mohamed Salah Hamdi. Search-based web service refactoring using quality of service and code quality metrics. *IEEE Transactions on Services Computing*, 10(4):636–649, 2017.

Shrideep Pallickara, Hasan Bulut, and Geoffrey Fox. Fault-tolerant reliable delivery of messages in distributed publish/subscribe systems. In *Proc. 4th IEEE International Conference on Autonomic Computing (ICAC)*, pages 12–21, 2007.

Riccardo Pinciroli, Aldeida Aleti, and Catia Trubiani. Performance modeling and analysis of design patterns for microservice systems. In *Proc. IEEE 20th International Conference on Software Architecture (ICSA)*, pages 35–46, 2023. doi: 10.1109/ICSA56044.2023.00012.

Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.

Thomas L. Saaty. *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. McGraw-Hill, 1980.

Tushar Sharma, Pratibha Mishra, and Rohit Tiwari. Designite: A software design quality assessment tool. In *Proc. 1st International Workshop on Bringing Architectural Design Thinking into Developers' Daily Activities (BRIDGE)*, pages 1–4, 2016. doi: 10.1145/2896935.2896938.

Connie U. Smith and Lloyd G. Williams. Software performance antipatterns. In *Proc. 2nd International Workshop on Software and Performance (WOSP)*, pages 127–136, 2000. doi: 10.1145/350391.350420.

Connie U. Smith and Lloyd G. Williams. New software performance AntiPatterns: More ways to shoot yourself in the foot. In *Proc. International Computer Measurement Group Conference (CMG)*, pages 667–674, 2002.

Cesar A. Stuardo, Tanakorn Leesatapornwongsa, Riza O. Suminto, Huan Ke, Jeffrey F. Lukman, Wei-Chiu Chuang, Shan Lu, and Haryadi S. Gunawi. ScaleCheck: A single-machine approach for discovering scalability bugs in large distributed systems. In *Proc. 17th USENIX Conference on File and Storage Technologies (FAST)*, pages 359–373, 2019.

Girish Suryanarayana, Ganesh Samarthyam, and Tushar Sharma. *Refactoring for Software Design Smells: Managing Technical Debt*. Morgan Kaufmann, 2014.

Davide Taibi, Valentina Lenarduzzi, and Claus Pahl. Microservices anti-patterns: A taxonomy. In *Microservices: Science and Engineering*, pages 211–228. Springer, 2020.

Catia Trubiani, Riccardo Pinciroli, Andrea Biaggi, and Francesca Arcelli Fontana. Automated detection of software performance antipatterns in Java-based applications. *IEEE Transactions on Software Engineering*, 49(4):2873–2891, 2023. doi: 10.1109/TSE.2023.3234321.

Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. Towards automating code review activities. *IEEE Transactions on Software Engineering*, 48 (8):3156–3173, 2022.

Guozhang Wang, Joel Koshy, Sriram Subramanian, Kartik Paramasivam, Mammad Zadeh, Neha Narkhede, Jun Rao, Jay Kreps, and Joe Stein. Building a replicated logging system with apache kafka. *Proceedings of the VLDB Endowment*, 8(12):1654–1655, 2015.

Colin White, Saurabh Agarwal, and Haoran Zhang. Toward an understanding of large language models for code refactoring. *IEEE Software*, 41(2):48–56, 2024.

Ibrahim Onuralp Yigit and Feza Buzluca. A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems. In *Proc. IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE)*, 2025.

Hao-Nan Zhu, Goodness Ayinmode, Cesar A. Stuardo, Haryadi S. Gunawi, and Cindy Rubio-González. Understanding and detecting scalability faults in large-scale distributed systems. *IEEE Transactions on Software Engineering*, 52(10):3166–3179, 2026. doi: 10.1109/TSE.2026.3710096.